

DELIMITED CONTINUATIONS DEMYSTIFIED

Alexis King, Tweag

Lambda Days 2023

HISTORY

HISTORY

→ Delimited continuations introduced by
Matthias Felleisen 35 years ago.

HISTORY

- Delimited continuations introduced by Matthias Felleisen 35 years ago.
- Flurry of initial publications, mostly in Scheme.

HISTORY

- Delimited continuations introduced by Matthias Felleisen 35 years ago.
- Flurry of initial publications, mostly in Scheme.
- Not much mainstream adoption.

HISTORY

- Delimited continuations introduced by Matthias Felleisen 35 years ago.
- Flurry of initial publications, mostly in Scheme.
- Not much mainstream adoption.
- Recently: some renewed interest.





→ Initial proposal in early 2020; revised version accepted in late 2020.

Haskell

- Initial proposal in early 2020; revised version accepted in late 2020.
- Implementation in limbo for several years.

Haskell

- Initial proposal in early 2020; revised version accepted in late 2020.
- Implementation in limbo for several years.
- Started at Tweag last year; patch landed last fall.

Haskell

- Initial proposal in early 2020; revised version accepted in late 2020.
- Implementation in limbo for several years.
- Started at Tweag last year; patch landed last fall.
- Finally released this past March in GHC 9.6!

Problem: nobody knows what they are.

DEMYSTIFICATION

TERMINOLOGY

TERMINOLOGY

“continuations”

TERMINOLOGY

~~// continuations //~~

// delimited continuations //

TERMINOLOGY

~~// continuations //~~

// first-class,
delimited continuations //

TERMINOLOGY

~~// continuations //~~

// native, first-class,
delimited continuations //

TERMINOLOGY

~~“continuations”~~

“native, first-class,
delimited continuations”

① continuations

② delimited

③ first-class

④ native

① continuations

② delimited

③ first-class

④ native

A “continuation” is a *concept*,
not a language feature.

A “continuation” is a *concept*,
not a language feature.

(Like “scope” or “value”.)

A “continuation” is a *concept*,
not a language feature.

(Like “scope” or “value”.)

Applies to most programming languages!

A “continuation” is a *concept*,
not a language feature.

(Like “scope” or “value”.)

Applies to most programming languages!

Useful for talking about *evaluation*.

$$(1 + 2) * (3 + 4)$$

$$(1 + 2) * (3 + 4)$$

$$\begin{array}{c} (1 + 2) * (3 + 4) \\ \downarrow \\ 3 * (3 + 4) \end{array}$$

$$(1 + 2) * (3 + 4)$$



$$3 * (3 + 4)$$

$$(1 + 2) * (3 + 4)$$



$$3 * (3 + 4)$$



$$3 * 7$$

$$(1 + 2) * (3 + 4)$$



$$3 * (3 + 4)$$



$$3 * 7$$

$$(1 + 2) * (3 + 4)$$



$$3 * (3 + 4)$$



$$3 * 7$$



$$21$$

$$(1 + 2) * (3 + 4)$$



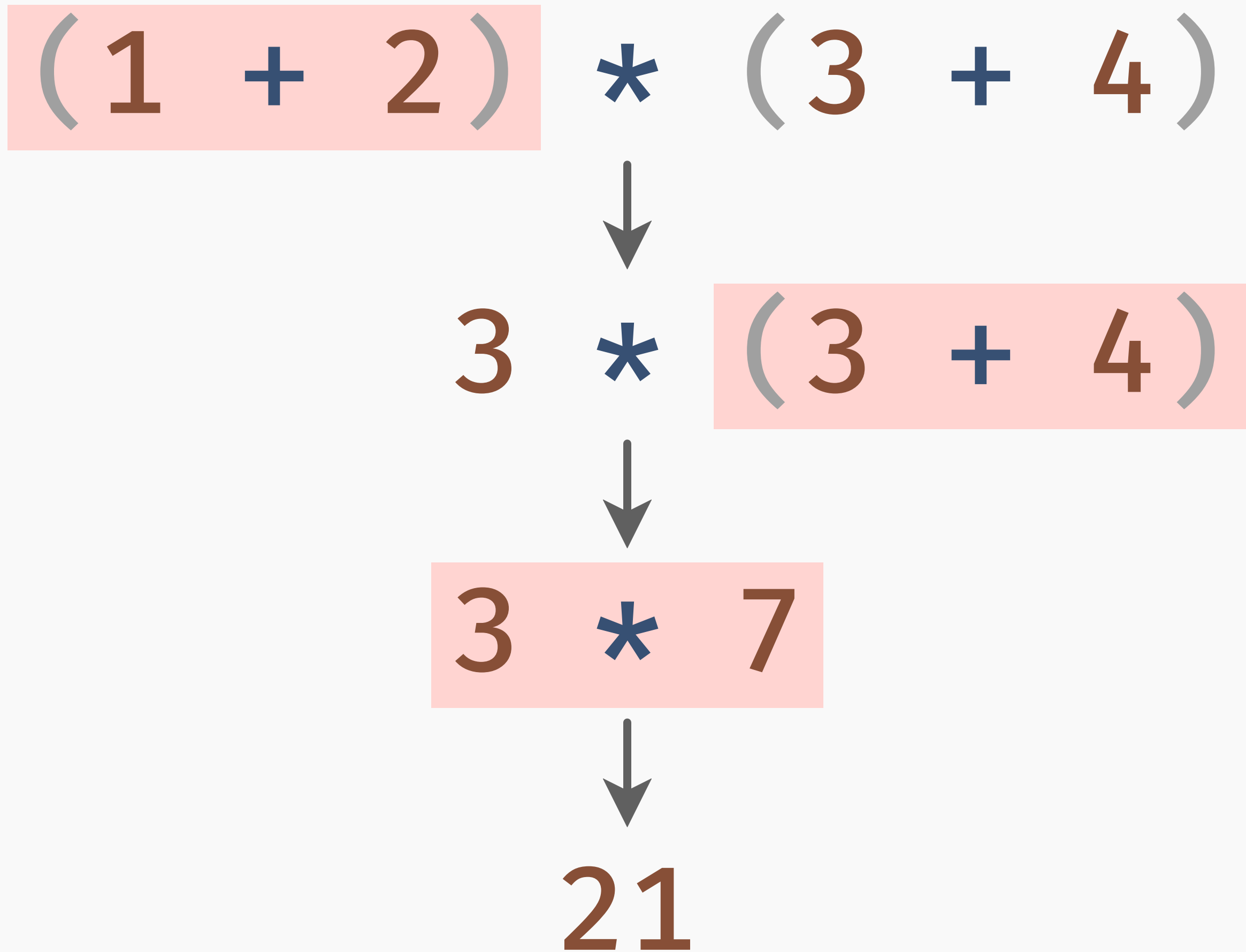
$$3 * (3 + 4)$$



$$3 * 7$$



$$21$$



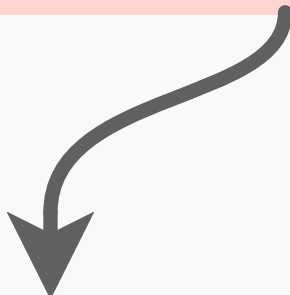
$$(1 + 2) * (3 + 4)$$

$$(1 + 2) * (3 + 4)$$

$$(1 + 2) * (3 + 4)$$

(1 + 2)

* (3 + 4)

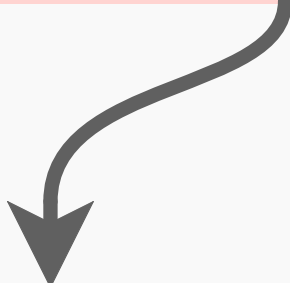


1 + 2

● * (3 + 4)

(1 + 2)

* (3 + 4)

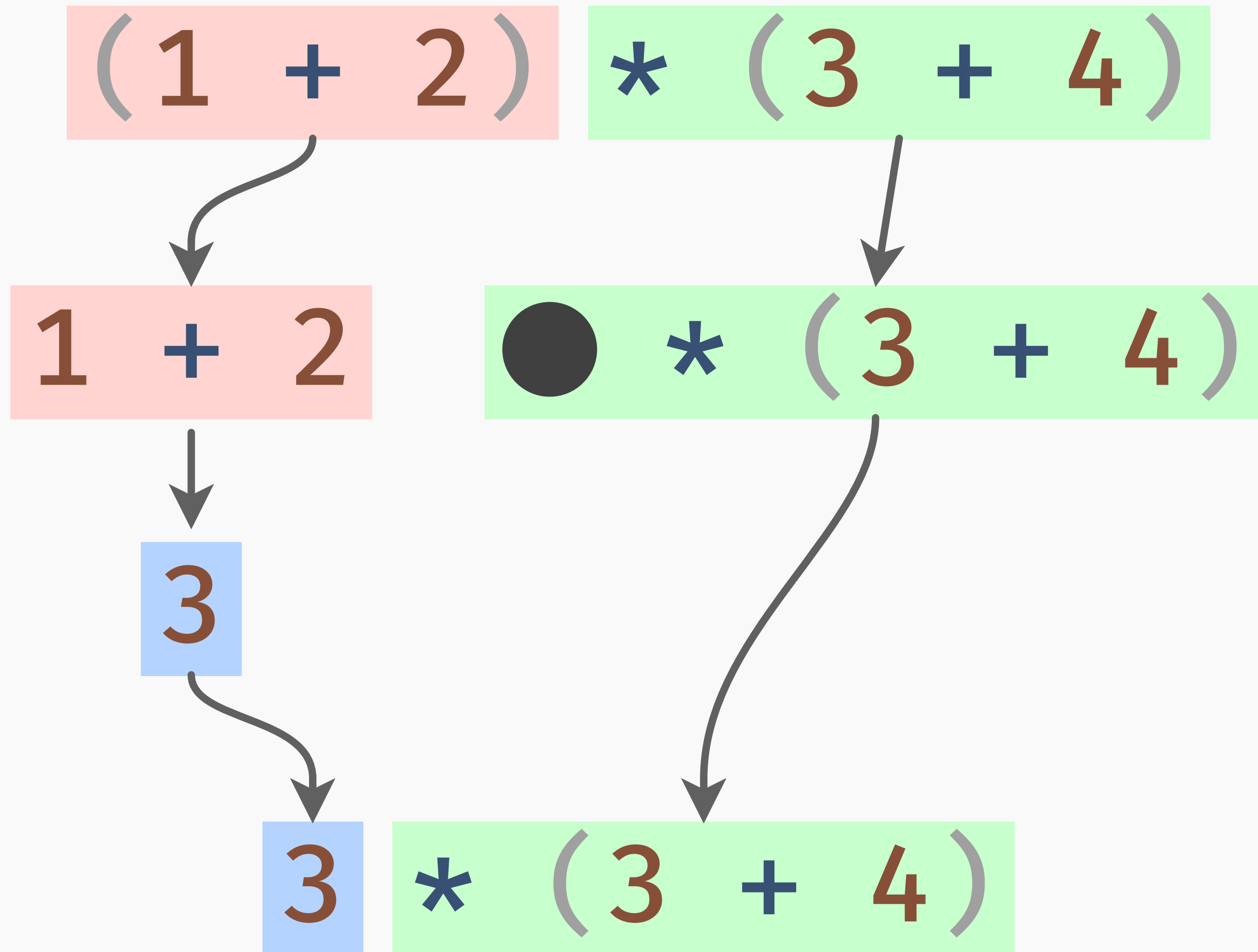


1 + 2

● * (3 + 4)



3



$$(1 + 2) * (3 + 4)$$

$$1 + 2 \quad \bullet * (3 + 4)$$

"redex"

$$3$$

$$3 * (3 + 4)$$

$(1 + 2)$

$* (3 + 4)$

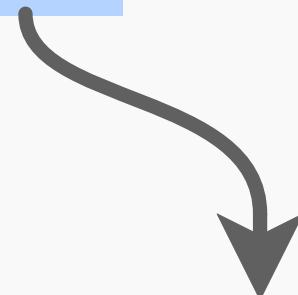
$1 + 2$

● $* (3 + 4)$

“redex”



3



3

$* (3 + 4)$



???

$(1 + 2) * (3 + 4)$

$1 + 2 \bullet * (3 + 4)$

“redex”

“continuation”

3

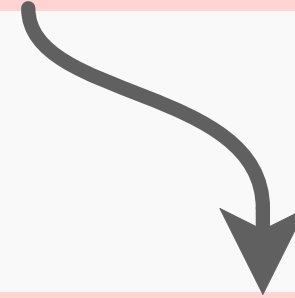
3 $* (3 + 4)$

$$3 * (3 + 4)$$

$$3 * (3 + 4)$$

3 *

(3 + 4)



3 * ●

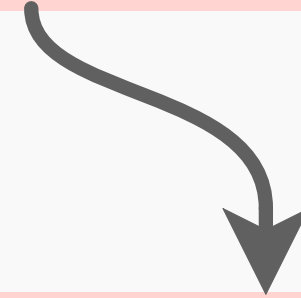
3 + 4

continuation

redex

3 *

(3 + 4)



3 * ●

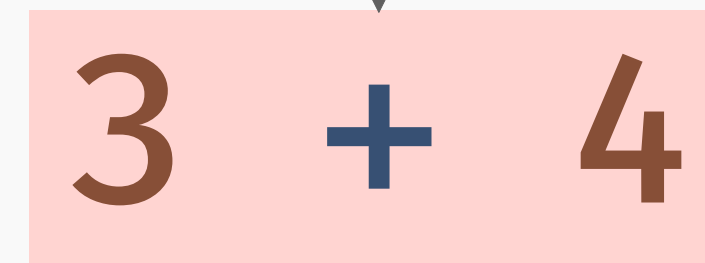
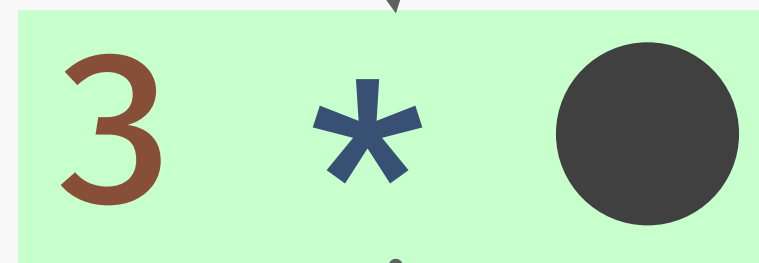
3 + 4



7

continuation

redex



continuation

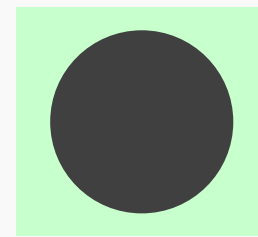
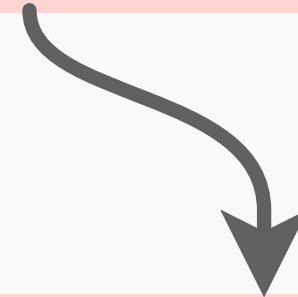
redex



$$3 * 7$$

$$3 * 7$$

$$3 * 7$$

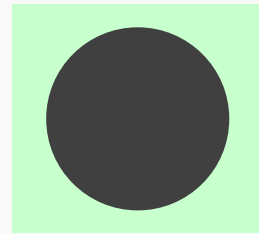


$$3 * 7$$

continuation
(empty)

redex

continuation
(empty)



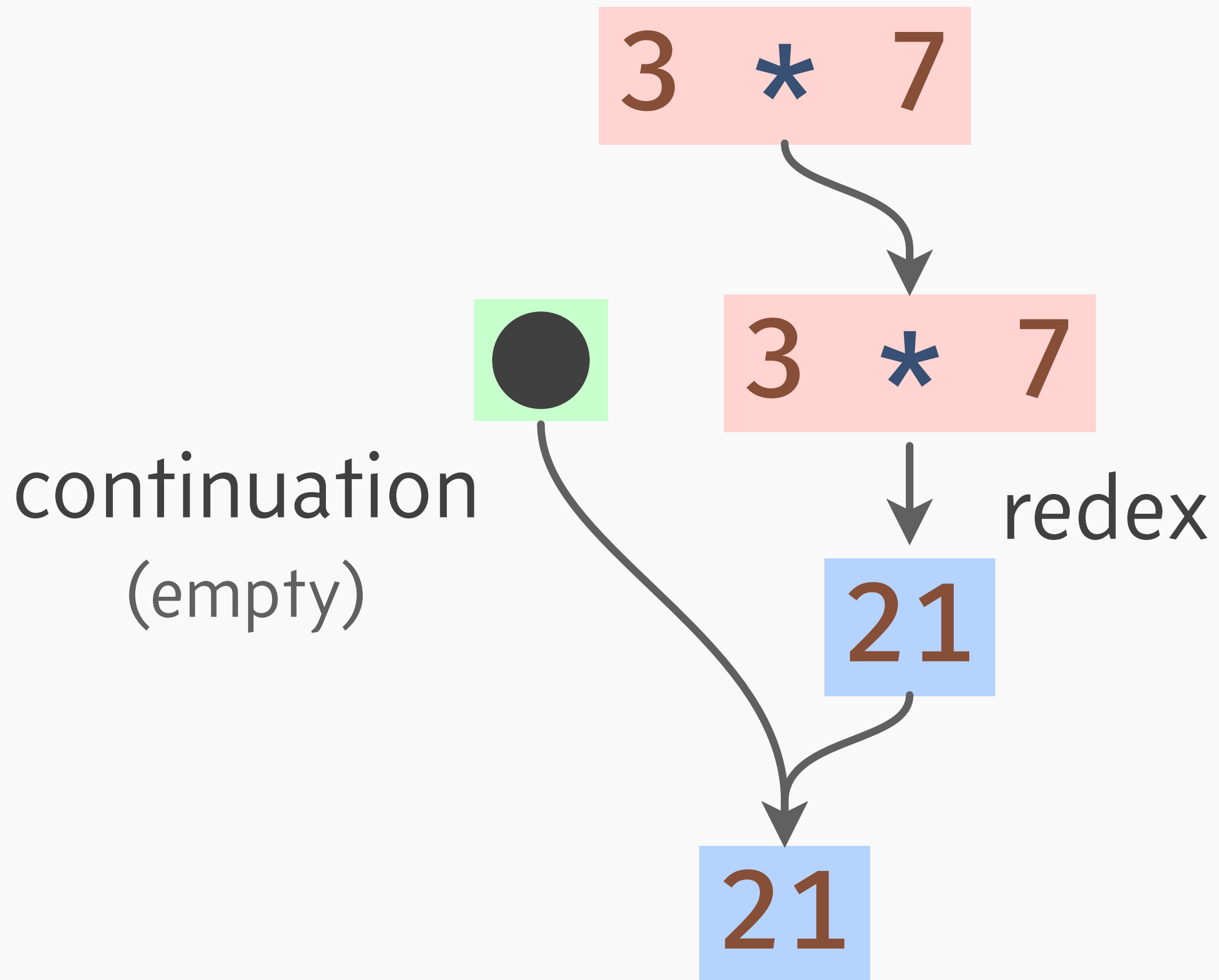
3 * 7

3 * 7



redex

21



What is the continuation?

What is the continuation?

→ The “context” in which the redex is evaluated.

What is the continuation?

- The “context” in which the redex is evaluated.
- An expression with a hole.

What is the continuation?

- The “context” in which the redex is evaluated.
- An expression with a hole.
- The place the redex’s value is “returned to”.

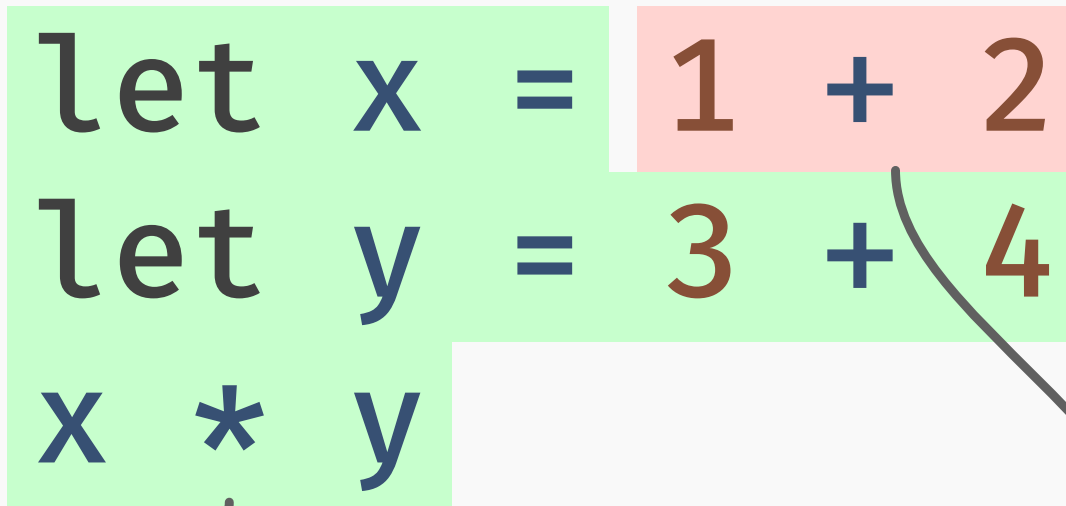
What is the continuation?

- The “context” in which the redex is evaluated.
- An expression with a hole.
- The place the redex’s value is “returned to”.
- “The rest of the program.”

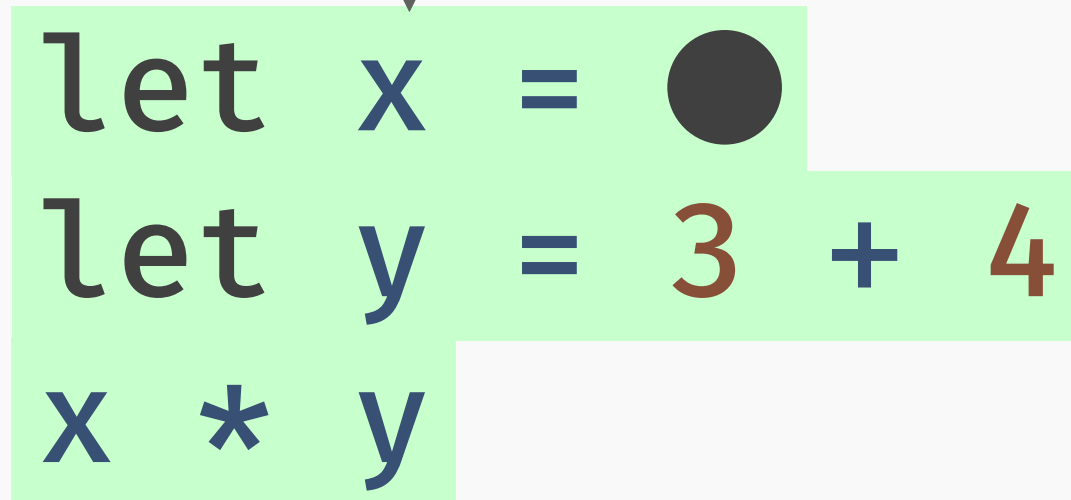
```
let x = 1 + 2
let y = 3 + 4
x * y
```

```
let x = 1 + 2  
let y = 3 + 4  
x * y
```

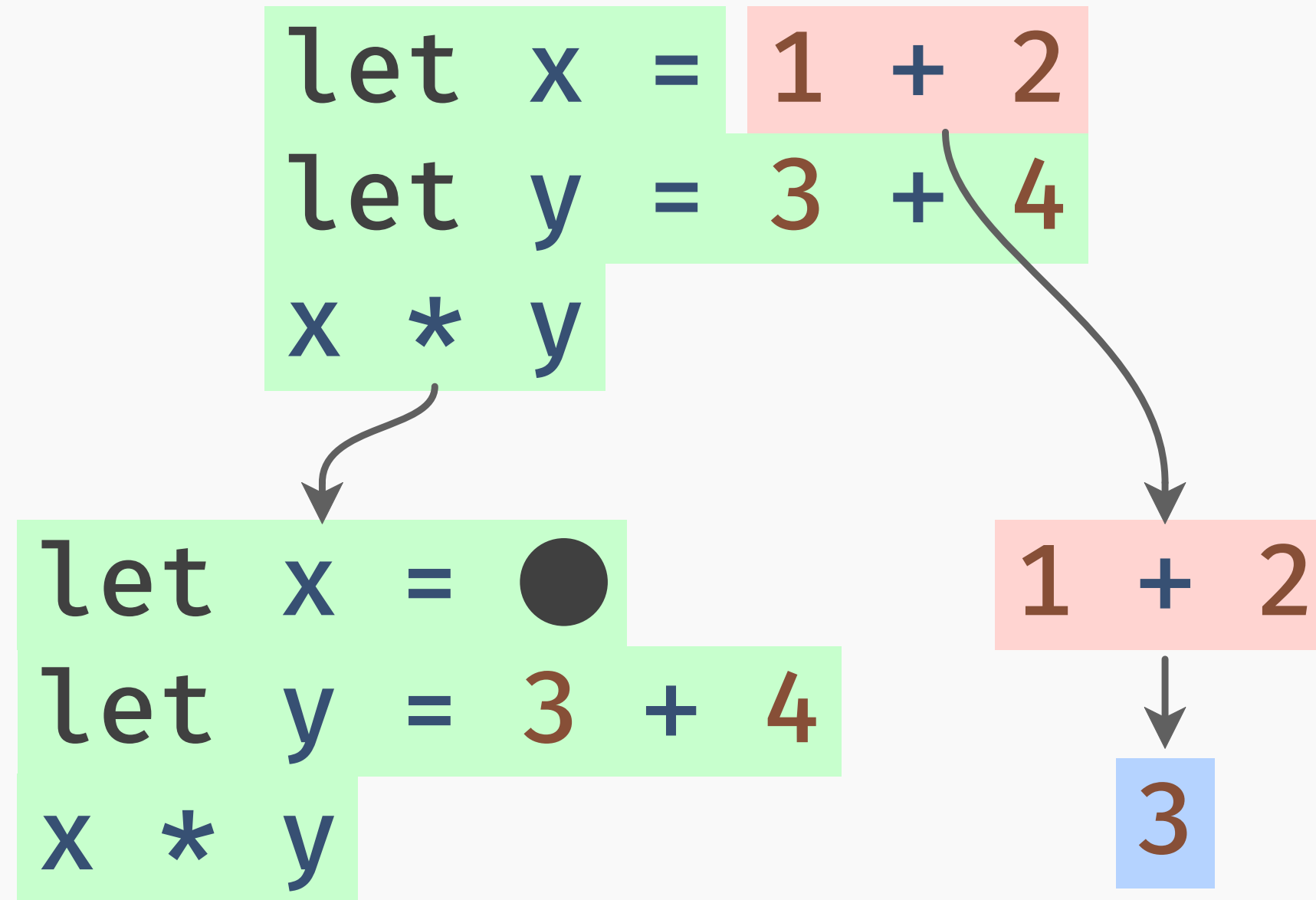
```
let x = 1 + 2  
let y = 3 + 4  
x * y
```

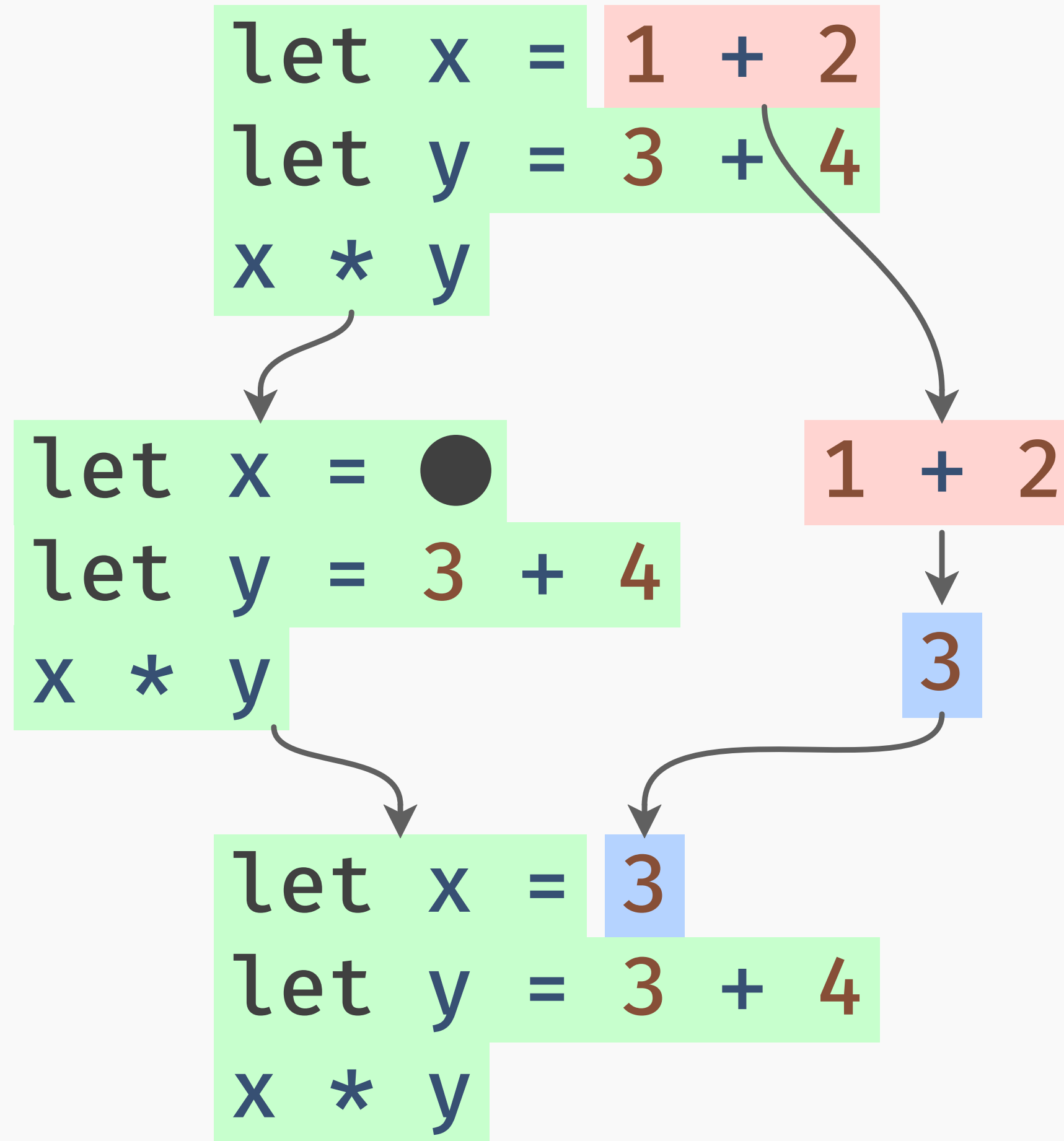


```
let x = ●  
let y = 3 + 4  
x * y
```



1 + 2





let $x = 3$

let $y = 3 + 4$

$x * y$

```
let x = 3
```

```
let y = 3 + 4
```

```
x * y
```

```
let x = 3
```

```
let y = 3 + 4
```

```
x * y
```

```
let x = 3
```

```
let y = 3 + 4
```

```
x * y
```



```
let y = 3 + 4
```

```
3 * y
```

```
let y = 3 + 4  
3 * y
```

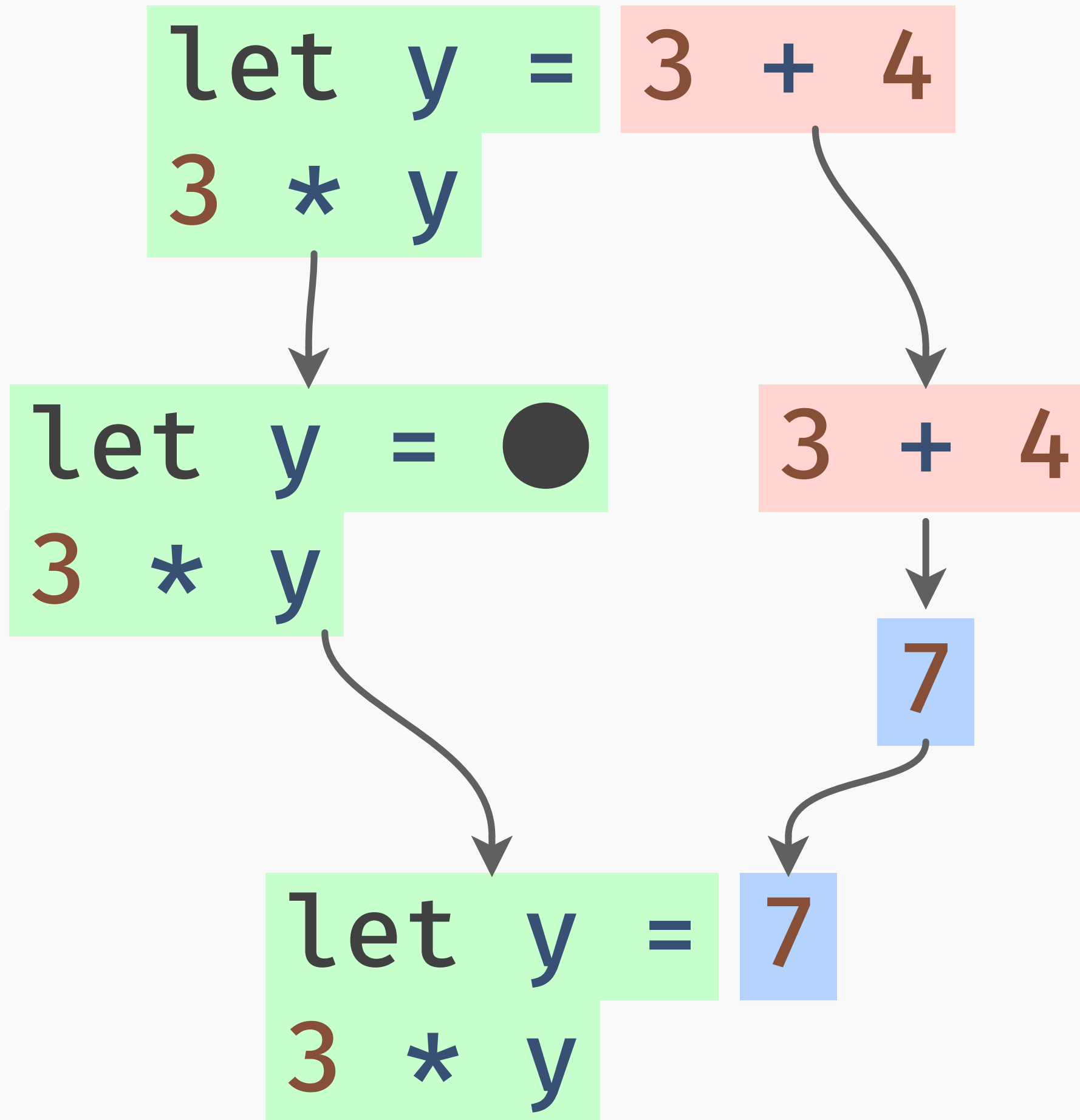
```
let y = 3 + 4  
3 * y
```


let y = 3 + 4
3 * y



let y = ●
3 * y

3 + 4



Why care about continuations?

Why care about continuations?

Evaluation is *extremely* regular:

Why care about continuations?

Evaluation is *extremely* regular:

- ① Split the redex and continuation.

Why care about continuations?

Evaluation is *extremely* regular:

- ① Split the redex and continuation.
- ② Reduce the redex.

Why care about continuations?

Evaluation is *extremely* regular:

- ① Split the redex and continuation.
- ② Reduce the redex.
- ③ Substitute the result into the continuation.

Why care about continuations?

Evaluation is *extremely* regular:

- ① Split the redex and continuation.
- ② Reduce the redex.
- ③ Substitute the result into the continuation.
- ④ Repeat.

Why care about continuations?

Evaluation is *extremely* regular:

- ① Split the redex and continuation.
- ② Reduce the redex.
- ③ Substitute the result into the continuation.
- ④ Repeat.

Why is the continuation itself interesting?

Compiler writers care about the continuation!

Compiler writers care about the continuation!

Most programmers don't have much
reason to, most of the time.

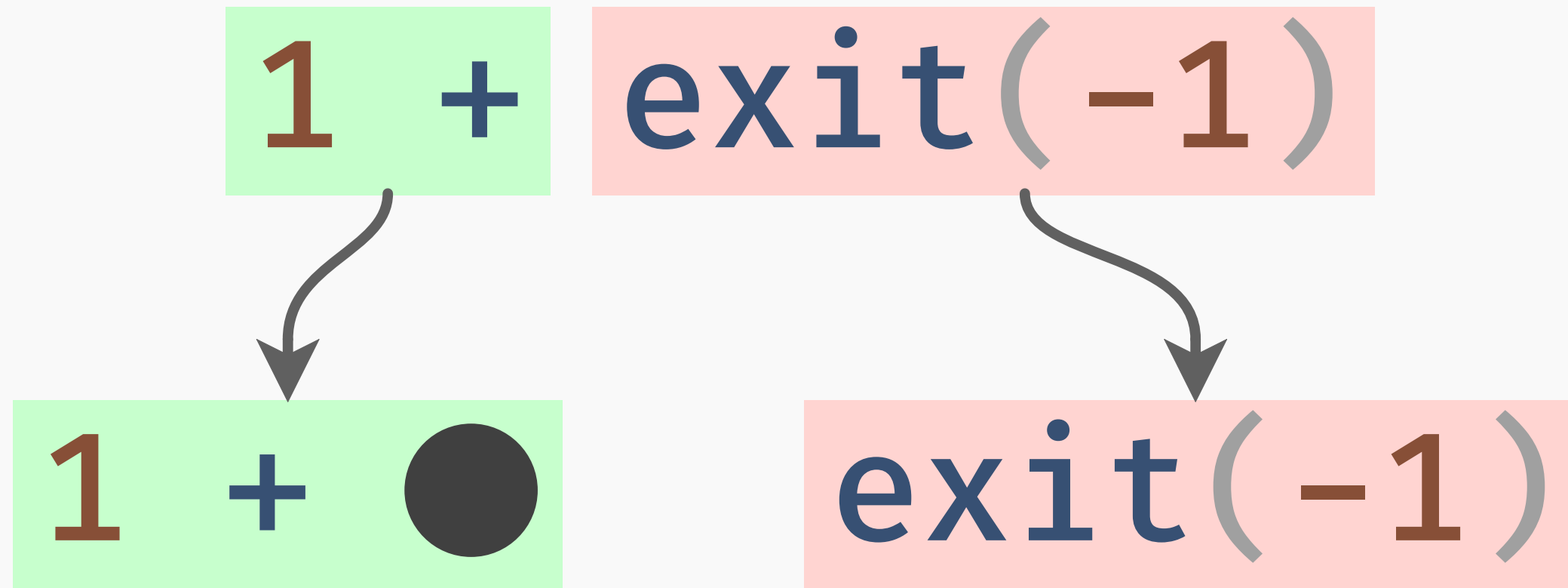
Compiler writers care about the continuation!

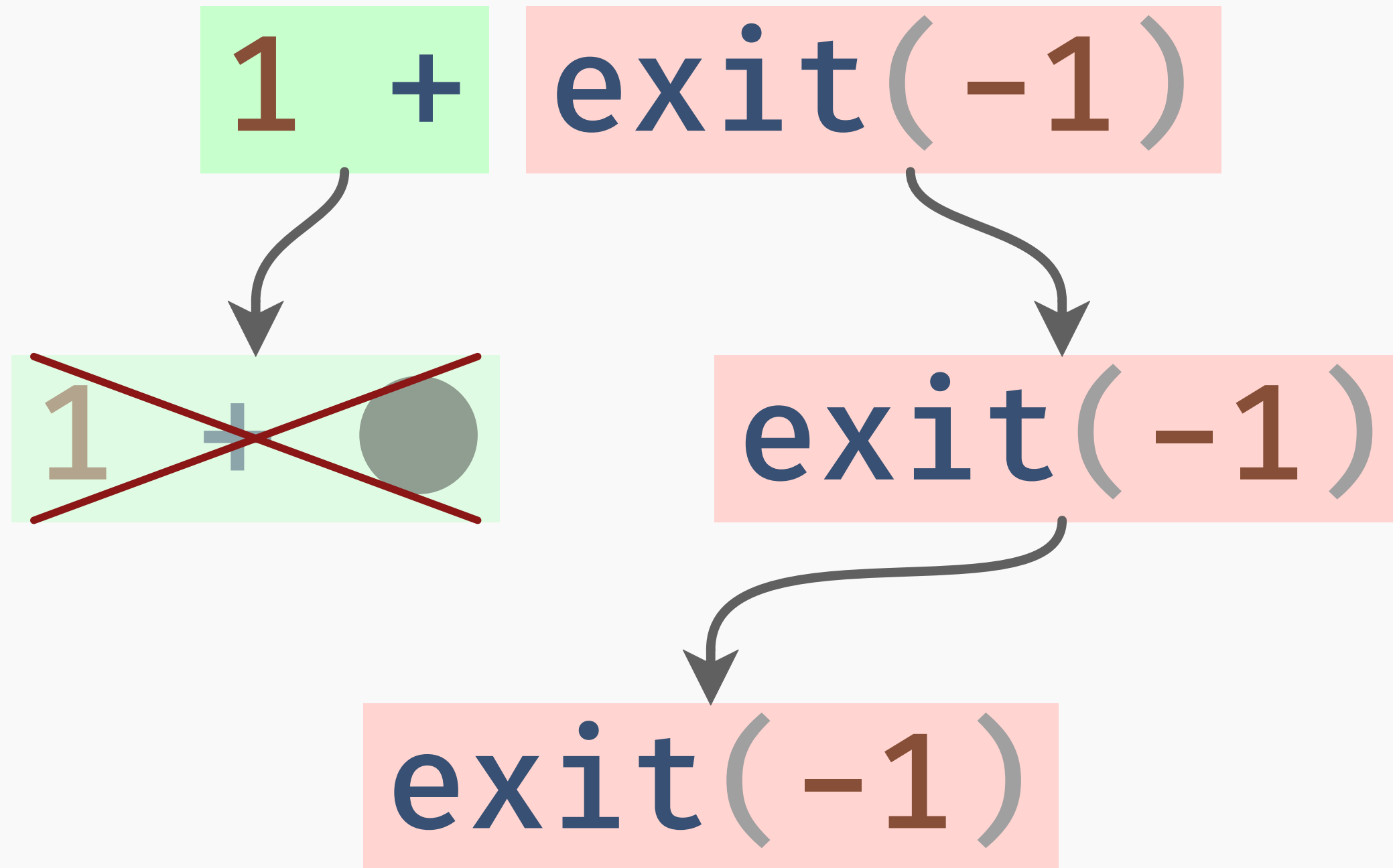
Most programmers don't have much
reason to, most of the time.

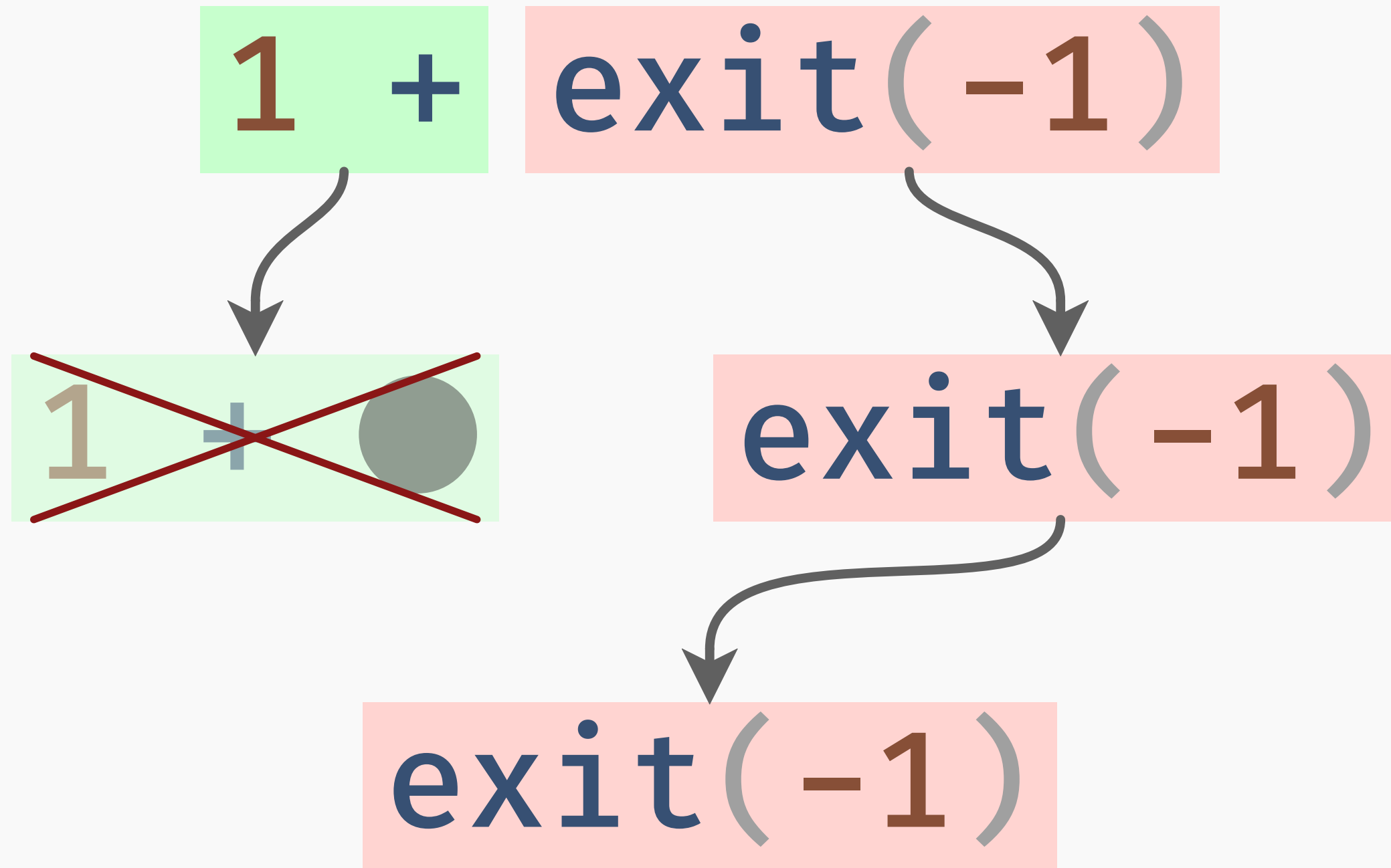
...but what about operators that use different rules?

1 + exit(-1)

```
1 + exit(-1)
```







Continuation is thrown away!

`exit` is still not terribly interesting.

What about **throw** / catch?

What about **throw** / catch?

throw(exn)

What about **throw** / **catch**?

throw(**exn**)

Raises **exn** as an exception.

What about **throw** / **catch**?

throw(**exn**)

Raises **exn** as an exception.

catch{**body**, **handler**}

What about **throw** / **catch**?

throw(**exn**)

Raises **exn** as an exception.

catch{**body**, **handler**}

Evaluates **body**, and if an exception is raised, evaluates **handler**(**exn**).

1 + catch { 2 * throw(5),
 (n) → 3 * n }

1 + catch { 2 * throw(5),
 (n) → 3 * n }

1 + catch { 2 * throw(5),
 (n) → 3 * n }

1 + catch{2 * throw(5),
(n) → 3 * n}

1 + catch{2 * throw(5),
(n) → 3 * n}



1 + (3 * 5)

$1 + \text{catch}\{2 * \text{throw}(5),$
 $\quad\quad\quad (n) \rightarrow 3 * n\}$

$\downarrow$

$1 + (3 * 5)$

$\downarrow$

$1 + 15$

1 + catch{2 * throw(5),
(n) → 3 * n}

↓
1 + (3 * 5)

↓
1 + 15

↓
16

1 + catch{2 * throw(5),
 (n) → 3 * n}

1 + catch{2 * throw(5),
(n) → 3 * n}

```
1 + catch{2 * throw(5),  
          (n) → 3 * n}
```


1 + catch{2 * throw(5),
(n) → 3 * n}



1 + catch{2 * ●,
(n) → 3 * n}



throw(5)

$1 + \text{catch}\{2 * \text{throw}(5),$
 $(n) \rightarrow 3 * n\}$

$1 + \text{catch}\{2 * \bullet,$
 $(n) \rightarrow 3 * n\}$

$\text{throw}(5)$

???

$1 + (3 * 5)$

1 + catch{2 * throw(5),
(n) → 3 * n}

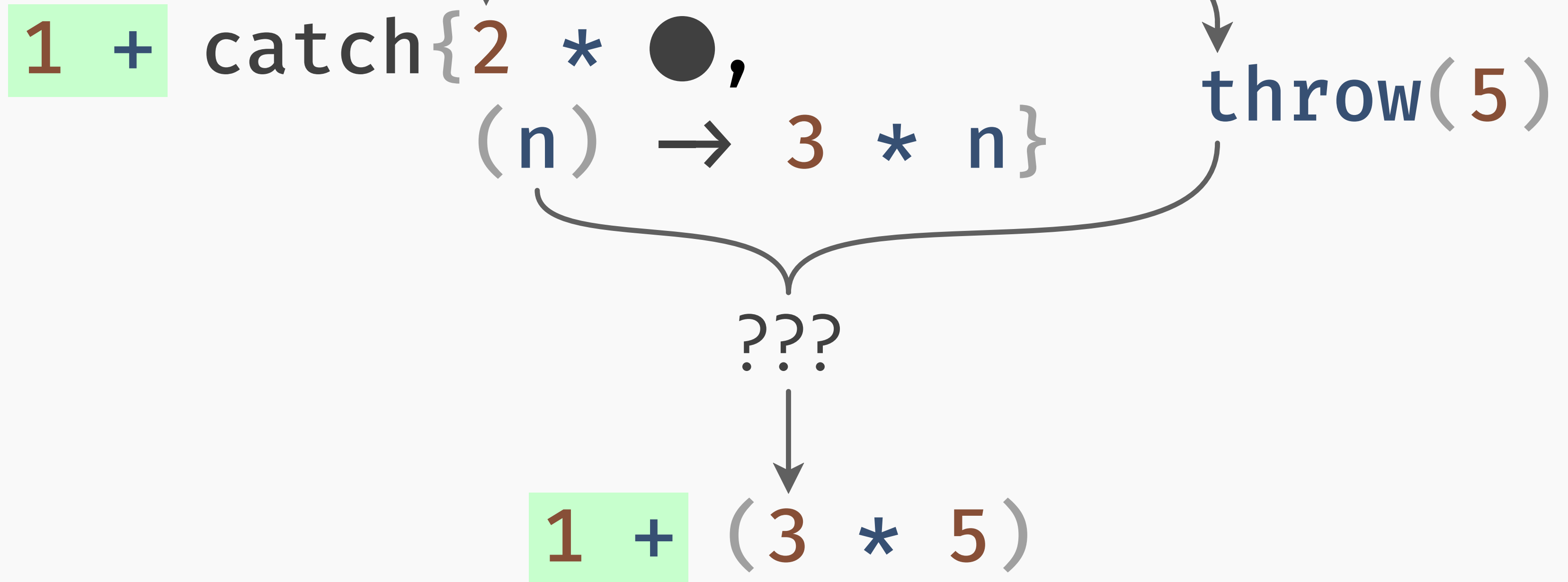
1 + catch{2 * ●,
(n) → 3 * n}

throw(5)

???

1 + (3 * 5)

1 + catch{2 * throw(5),
(n) → 3 * n}



1 + catch{2 * throw(5),
(n) → 3 * n}

1 + catch{2 * ●,
(n) → 3 * n} throw(5)

???

1 + (3 * 5)

1 + catch{2 * throw(5),
(n) → 3 * n}

1 + catch{~~2 *~~ ●,
(n) → 3 * n} throw(5)

???

1 + (3 * 5)

1 + catch{2 * throw(5),
(n) → 3 * n}

1 + catch{2 * ●,
(n) → 3 * n}

throw(5)



???

1 + (3 * 5)

1 + catch{2 * ●, (n) → 3 * n}

1 + catch{2 * ●, (n) → 3 * n}

$1 + \text{catch}\{2 * \bullet, (n) \rightarrow 3 * n\}$

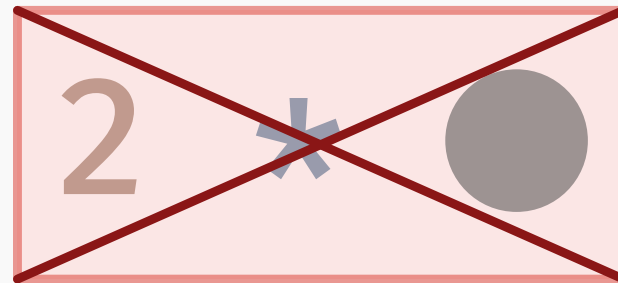
$1 + \text{catch}\{2 * \bullet, (n) \rightarrow 3 * n\}$

$2 * \bullet$

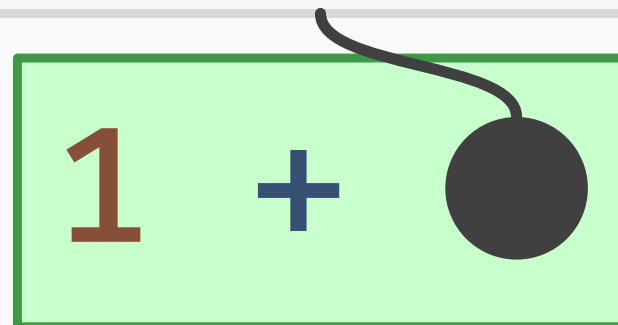
$\text{catch}\{\bullet, (n) \rightarrow 3 * n\}$

$1 + \bullet$

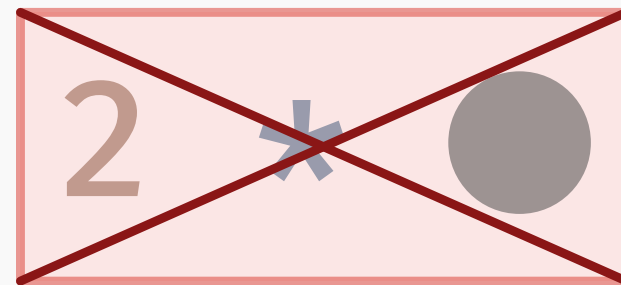
$1 + \text{catch}\{2 * \bullet, (n) \rightarrow 3 * n\}$



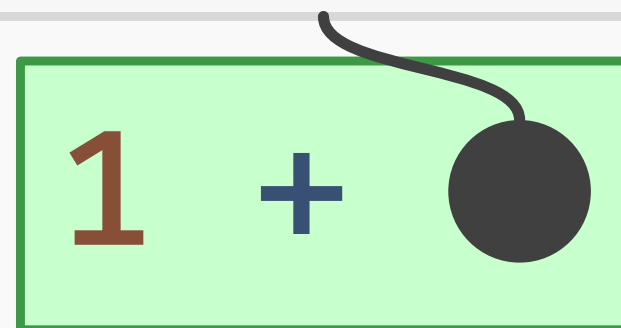
$\text{catch}\{\bullet, (n) \rightarrow 3 * n\}$



$1 + \text{catch}\{2 * \bullet, (n) \rightarrow 3 * n\}$



$\text{catch}\{\bullet, (n) \rightarrow 3 * n\}$



catch delimits the discarded continuation.

INTERLUDE: NOTATION

$$A \longrightarrow B$$

$$A \longrightarrow B$$

" A reduces to B ."

$$A \longrightarrow B$$

" A reduces to B ."

`not(false) → true`

$$A \longrightarrow B$$

" A reduces to B ."

`not(false)` $\longrightarrow$ `true`

`not(true)` $\longrightarrow$ `false`

$$A \longrightarrow B$$

" A reduces to B ."

$$\text{not}(\text{false}) \longrightarrow \text{true}$$

$$\text{not}(\text{true}) \longrightarrow \text{false}$$

$$\text{if } \text{true} \quad \text{then } e_1 \text{ else } e_2 \longrightarrow e_1$$

$$A \longrightarrow B$$

" A reduces to B ."

$$\text{not}(\text{false}) \longrightarrow \text{true}$$

$$\text{not}(\text{true}) \longrightarrow \text{false}$$

$$\text{if true then } e_1 \text{ else } e_2 \longrightarrow e_1$$

$$\text{if false then } e_1 \text{ else } e_2 \longrightarrow e_2$$

$$A \longrightarrow B$$

" A reduces to B ."

$$\text{not}(\text{false}) \longrightarrow \text{true}$$

$$\text{not}(\text{true}) \longrightarrow \text{false}$$

$$\text{if true then } e_1 \text{ else } e_2 \longrightarrow e_1$$

$$\text{if false then } e_1 \text{ else } e_2 \longrightarrow e_2$$

$$\text{if not}(\text{false}) \text{ then } 1 \text{ else } 2?$$

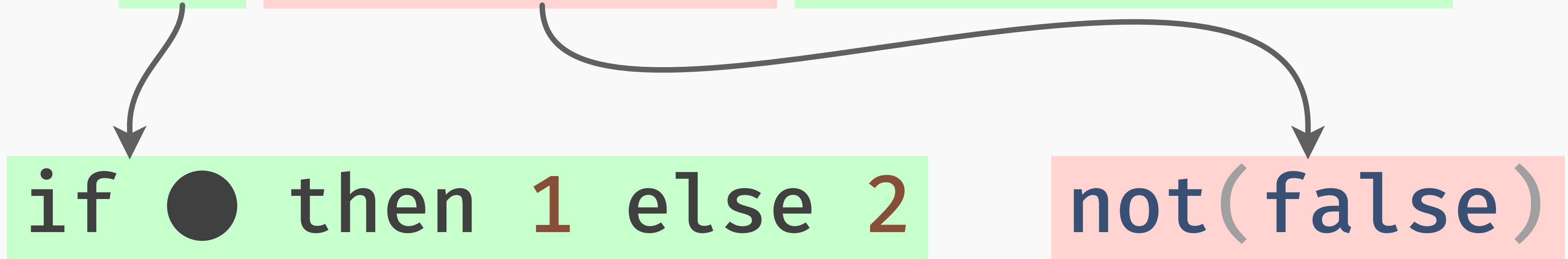

```
if not(false) then 1 else 2
```

```
if not(false) then 1 else 2
```

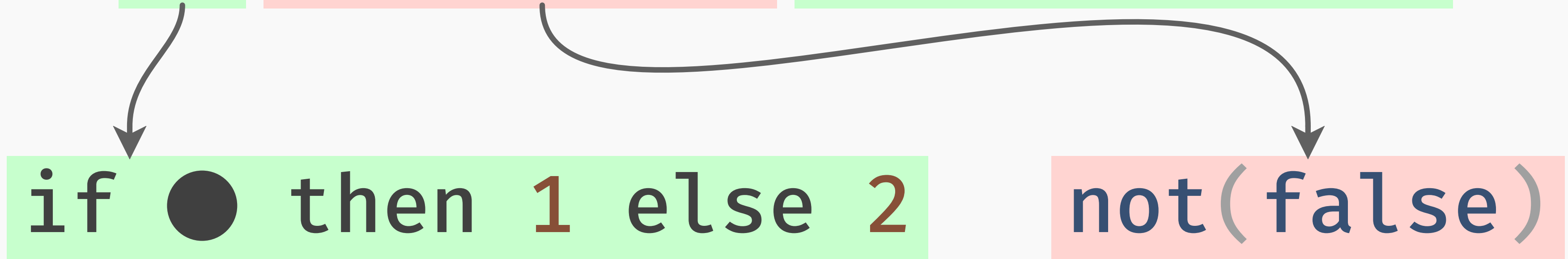


```
if not(false) then 1 else 2
```

if not(false) then 1 else 2

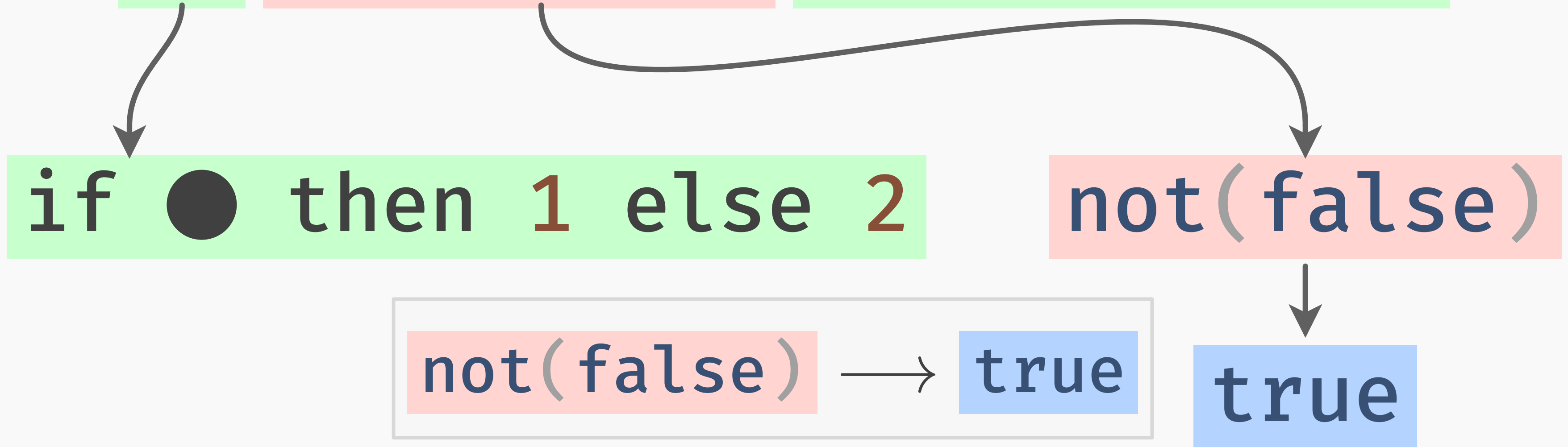


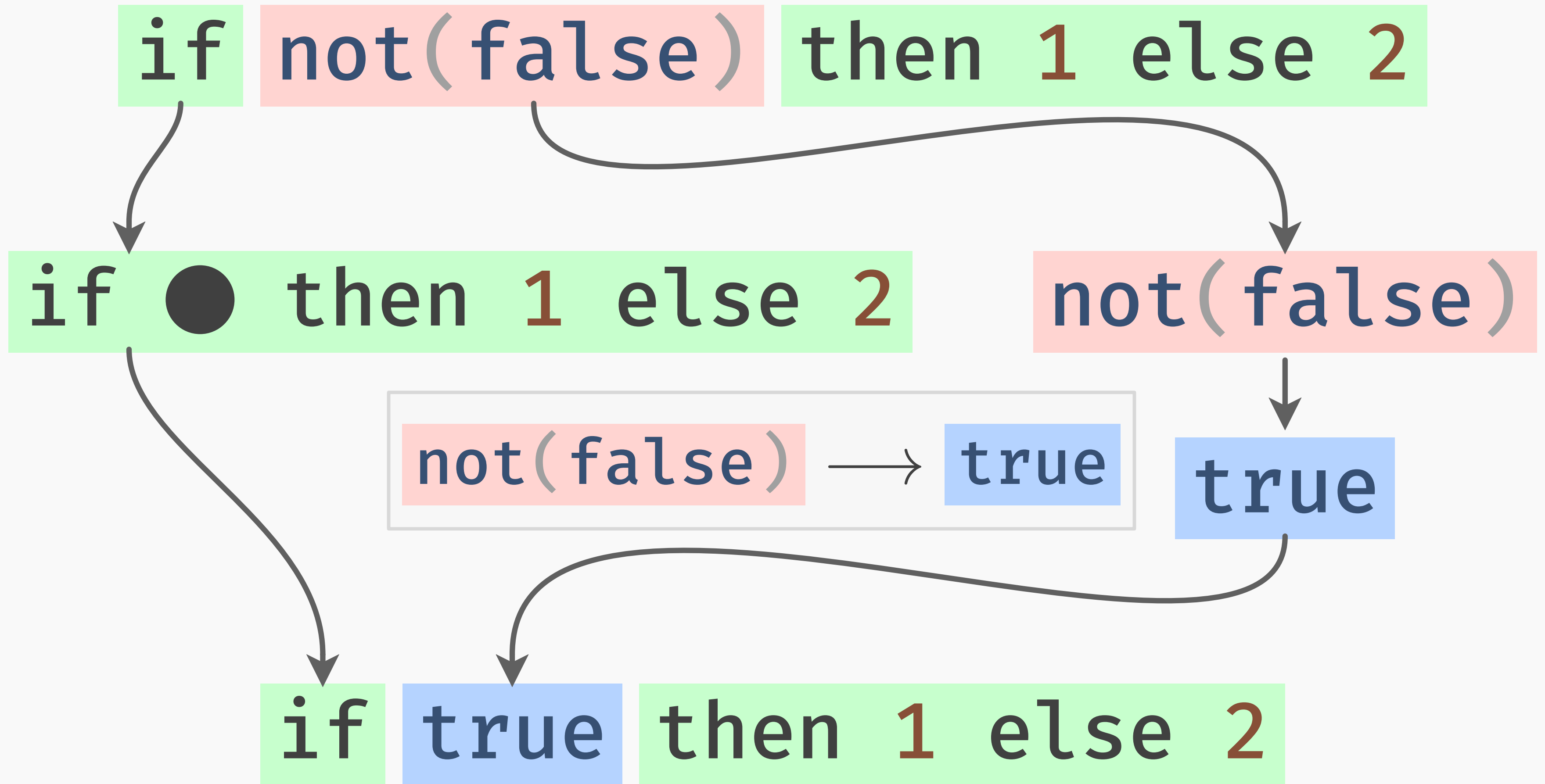
if not(false) then 1 else 2



not(false) → true

if not(false) then 1 else 2





`not(false) → true`

$$\begin{array}{ccc} \text{not}(\text{false}) & \longrightarrow & \text{true} \\ E[\text{not}(\text{false})] & \longrightarrow & E[\text{true}] \end{array}$$

$$\begin{array}{c} \text{not}(\text{false}) \longrightarrow \text{true} \\ E[\text{not}(\text{false})] \longrightarrow E[\text{true}] \end{array}$$

→ E stands for “some arbitrary continuation”.

$$\begin{array}{c} \text{not}(\text{false}) \longrightarrow \text{true} \\ E[\text{not}(\text{false})] \longrightarrow E[\text{true}] \end{array}$$

- E stands for “some arbitrary continuation”.
- $E[x]$ denotes “plugging the hole” in E with x .

$$\frac{\text{not}(\text{false}) \longrightarrow \text{true}}{E[\text{not}(\text{false})] \longrightarrow E[\text{true}]}$$

- E stands for “some arbitrary continuation”.
- $E[x]$ denotes “plugging the hole” in E with x .

$E = \text{if } \bullet \text{ then } 1 \text{ else } 2$

$$\text{not}(\text{false}) \longrightarrow \text{true}$$

$$E[\text{not}(\text{false})] \longrightarrow E[\text{true}]$$

- E stands for “some arbitrary continuation”.
- $E[x]$ denotes “plugging the hole” in E with x .

$E = \text{if } \bullet \text{ then } 1 \text{ else } 2$

$x = \text{not}(\text{false})$

$$\frac{\text{not}(\text{false}) \longrightarrow \text{true}}{E[\text{not}(\text{false})] \longrightarrow E[\text{true}]}$$

- E stands for “some arbitrary continuation”.
- $E[x]$ denotes “plugging the hole” in E with x .

$E = \text{if } \bullet \text{ then } 1 \text{ else } 2$

$x = \text{not}(\text{false})$

$E[x] = \text{if } \text{not}(\text{false}) \text{ then } 1 \text{ else } 2$

Why bother with all of this?

Why bother with all of this?

$$E[\text{exit}(v)] \longrightarrow \text{exit}(v)$$

Why bother with all of this?

$$E[\text{exit}(v)] \longrightarrow \text{exit}(v)$$

Why bother with all of this?

$$E[\mathbf{exit}(v)] \longrightarrow \mathbf{exit}(v)$$

$$E_1[\mathbf{catch}\{E_2[\mathbf{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

Why bother with all of this?

$$E[\mathbf{exit}(v)] \longrightarrow \mathbf{exit}(v)$$

$$E_1[\mathbf{catch}\{E_2[\mathbf{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

Why bother with all of this?

$$E[\mathbf{exit}(v)] \longrightarrow \mathbf{exit}(v)$$

$$E_1[\mathbf{catch}\{E_2[\mathbf{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

Why bother with all of this?

$$E[\mathbf{exit}(v)] \longrightarrow \mathbf{exit}(v)$$

$$E_1[\mathbf{catch}\{E_2[\mathbf{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

Why bother with all of this?

$$E[\mathbf{exit}(v)] \longrightarrow \mathbf{exit}(v)$$

$$E_1[\mathbf{catch}\{E_2[\mathbf{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

Lots of operations can be described this way!

① continuations

② delimited

③ first-class

④ native

① continuations ✓

② delimited

③ first-class

④ native

① continuations ✓

② delimited ✓

③ first-class

④ native

① continuations ✓

② delimited ✓

③ first-class

④ native

What makes something “first class”?

How could a *continuation* be a *value*?

1 + (● * 2)

if ● > 0 then 1 else -1

f(catch{throw(●), handle})

1 + (● * 2)

if ● > 0 then 1 else -1

f(catch{throw(●), handle})

1 + (x * 2)

if x > 0 then 1 else -1

f(catch{throw(x), handle})

$(x) \rightarrow 1 + (x * 2)$

$(x) \rightarrow \text{if } x > 0 \text{ then } 1 \text{ else } -1$

$(x) \rightarrow f(\text{catch}\{\text{throw}(x), \text{handle}\})$

$(x) \rightarrow 1 + (x * 2)$

$(x) \rightarrow \text{if } x > 0 \text{ then } 1 \text{ else } -1$

$(x) \rightarrow f(\text{catch}\{\text{throw}(x), \text{handle}\})$

What is a “first-class continuation”?

What is a “first-class continuation”?

Answer: a continuation reified as a function.

call_cc

`call_cc`

“call with current continuation”

call_cc

“call with current continuation”

$$E[\text{call_cc}(f)] \longrightarrow E[f((x) \rightarrow E[x])]$$

call_cc

“call with current continuation”

$$E[\text{call_cc}(f)] \longrightarrow E[f((x) \rightarrow E[x])]$$

call_cc

“call with current continuation”

$$E[\text{call_cc}(f)] \longrightarrow E[f((x) \rightarrow E[x])]$$

call_cc

“call with current continuation”

$$E[\text{call_cc}(f)] \longrightarrow E[f((x) \rightarrow E[x])]$$

call_cc

“call with current continuation”

$$E[\text{call_cc}(f)] \longrightarrow E[f((x) \rightarrow E[x])]$$

This has some problems!

$$1 + (\bullet * 2)$$

~~1 + (● * 2)~~

```
print(1 + (● * 2))  
shutdown_runtime()  
run_libc_atexit()  
exit_process()
```

We need more control!

We need more control!

prompt / control

We need more control!

prompt / control

$$E_1[\text{prompt}\{E_2[\text{control}(f)]\}]$$

$$\longrightarrow E_1[f((\mathbf{x}) \longrightarrow E_2[\mathbf{x}])]$$

We need more control!

prompt / control

$$E_1[\text{prompt}\{E_2[\text{control}(f)]\}]$$

$$\longrightarrow E_1[f((x) \rightarrow E_2[x])]$$

We need more control!

prompt / control

$$E_1[\text{prompt} \{ E_2[\text{control}(f)] \}]$$

$$\longrightarrow E_1[f((x) \rightarrow E_2[x])]$$

We need more control!

prompt / control

$$E_1[\text{prompt}\{E_2[\text{control}(f)]\}]$$

$$\longrightarrow E_1[f((x) \longrightarrow E_2[x])]$$

We need more control!

prompt / control

$$E_1[\text{prompt} \{ E_2[\text{control}(f)] \}]$$

$$\longrightarrow E_1[f((x) \longrightarrow E_2[x])]$$

1 + prompt{2 * control((k) → k(3) + k(5))}

1 + prompt{2 * control((k) → k(3) + k(5))}

1 + prompt { 2 * control((k) → k(3) + k(5))) }

1 + prompt { 2 * control((k) → k(3) + k(5)) }



1 + prompt { **2** * control((k) → k(**3**) + k(**5**)) }

1 + ●

2 * ●

(k) → k(**3**) + k(**5**)

let k = (x) → **2** * x

k(**3**) + k(**5**)

1 + prompt { **2** * control((k) → k(**3**) + k(**5**)) }

1 + ●

2 * ●

(k) → k(**3**) + k(**5**)

let k = (x) → **2** * x

k(**3**) + k(**5**)

1 +

(let k = (x) → **2** * x
k(**3**) + k(**5**))

1 + prompt{2 * control((k) → k(3) + k(5))}



1 + (let k = (x) → 2 * x
k(3) + k(5))

1 + prompt{2 * control((k) → k(3) + k(5))}



1 + (let k = (x) → 2 * x
k(3) + k(5))

1 + prompt{2 * control((k) → k(3) + k(5))}



1 + (let k = (x) → 2 * x
k(3) + k(5))

1 + prompt{2 * control((k) → k(3) + k(5))}



1 + (let k = (x) → 2 * x
k(3) + k(5))

1 + prompt{2 * control((k) → k(3) + k(5))}

↓

1 + (let k = (x) → 2 * x
k(3) + k(5))

↓

1 + (6 + 10)

1 + prompt{2 * control((k) → k(3) + k(5))}



1 + (let k = (x) → 2 * x
k(3) + k(5))



1 + (6 + 10)



1 + 16

1 + prompt{2 * control((k) → k(3) + k(5))}



1 + (let k = (x) → 2 * x
k(3) + k(5))



1 + (6 + 10)



1 + 16



17

Why is this so confusing?

Why is this so confusing?

$$E_1[\text{catch}\{E_2[\text{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

$$E_1[\text{prompt}\{E_2[\text{control}(f)]\}] \longrightarrow E_1[f((\mathbf{x}) \rightarrow E_2[\mathbf{x}])]$$

Why is this so confusing?

$$E_1[\text{catch}\{E_2[\text{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

$$E_1[\text{prompt}\{E_2[\text{control}(f)]\}] \longrightarrow E_1[f((x) \rightarrow E_2[x])]$$

Why is this so confusing?

$$E_1[\text{catch}\{E_2[\text{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

$$E_1[\text{prompt}\{E_2[\text{control}(f)]\}] \longrightarrow E_1[f((x) \rightarrow E_2[x])]$$

Why is this so confusing?

$$E_1[\text{catch}\{E_2[\text{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

$$E_1[\text{prompt}\{E_2[\text{control}(f)]\}] \longrightarrow E_1[f((\mathbf{x}) \rightarrow E_2[\mathbf{x}])]$$

Why is this so confusing?

$$E_1[\text{catch}\{E_2[\text{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

$$E_1[\text{prompt}\{E_2[\text{control}(f)]\}] \longrightarrow E_1[f((x) \rightarrow E_2[x])]$$

Why is this so confusing?

$$E_1[\text{catch}\{E_2[\text{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

$$E_1[\text{prompt}\{E_2[\text{control}(f)]\}] \longrightarrow E_1[f((x) \rightarrow E_2[x])]$$

Why is this so confusing?

$$E_1[\text{catch}\{E_2[\text{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

$$E_1[\text{prompt}\{E_2[\text{control}(f)]\}] \longrightarrow E_1[f((x) \rightarrow E_2[x])]$$

Why is this so confusing?

$$E_1[\text{catch}\{E_2[\text{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

$$E_1[\text{prompt}\{E_2[\text{control}(f)]\}] \longrightarrow E_1[f((x) \rightarrow E_2[x])]$$

$$E_1[\text{delimit}\{E_2[\text{yield}(v)], f\}] \longrightarrow E_1[f(v, (x) \rightarrow E_2[x])]$$

Why is this so confusing?

$$E_1[\text{catch}\{E_2[\text{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

$$E_1[\text{prompt}\{E_2[\text{control}(f)]\}] \longrightarrow E_1[f((x) \rightarrow E_2[x])]$$

$$E_1[\text{delimit}\{E_2[\text{yield}(v)], f\}] \longrightarrow E_1[f(v, (x) \rightarrow E_2[x])]$$

Why is this so confusing?

$$E_1[\text{catch}\{E_2[\text{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

$$E_1[\text{prompt}\{E_2[\text{control}(f)]\}] \longrightarrow E_1[f((x) \rightarrow E_2[x])]$$

$$E_1[\text{delimit}\{E_2[\text{yield}(v)], f\}] \longrightarrow E_1[f(v, (x) \rightarrow E_2[x])]$$

Why is this so confusing?

$$E_1[\text{catch}\{E_2[\text{throw}(v)], f\}] \longrightarrow E_1[f(v)]$$

$$E_1[\text{prompt}\{E_2[\text{control}(f)]\}] \longrightarrow E_1[f((x) \rightarrow E_2[x])]$$

$$E_1[\text{delimit}\{E_2[\text{yield}(v)], f\}] \longrightarrow E_1[f(v, (x) \rightarrow E_2[x])]$$

`delimit` / `yield` provide *resumable exceptions*.

```
1 + delimit{2 * yield(()),  
            ((), k) → k(3) + k(5)}
```

```
1 + delimit{2 * yield(()),  
            ((), k) → k(3) + k(5)}
```

```
1 + delimit{2 * yield(()),  
            ((), k) → k(3) + k(5)}
```



```
1 + delimit{2 * yield(()),  
            ((), k) → k(3) + k(5)}
```

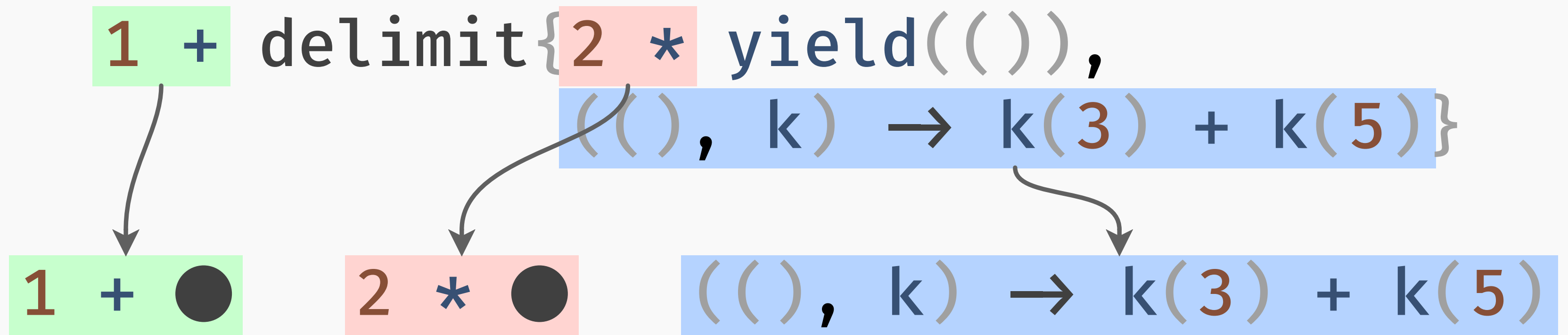
```
1 + delimit{2 * yield(()),  
            ((), k) → k(3) + k(5)}
```

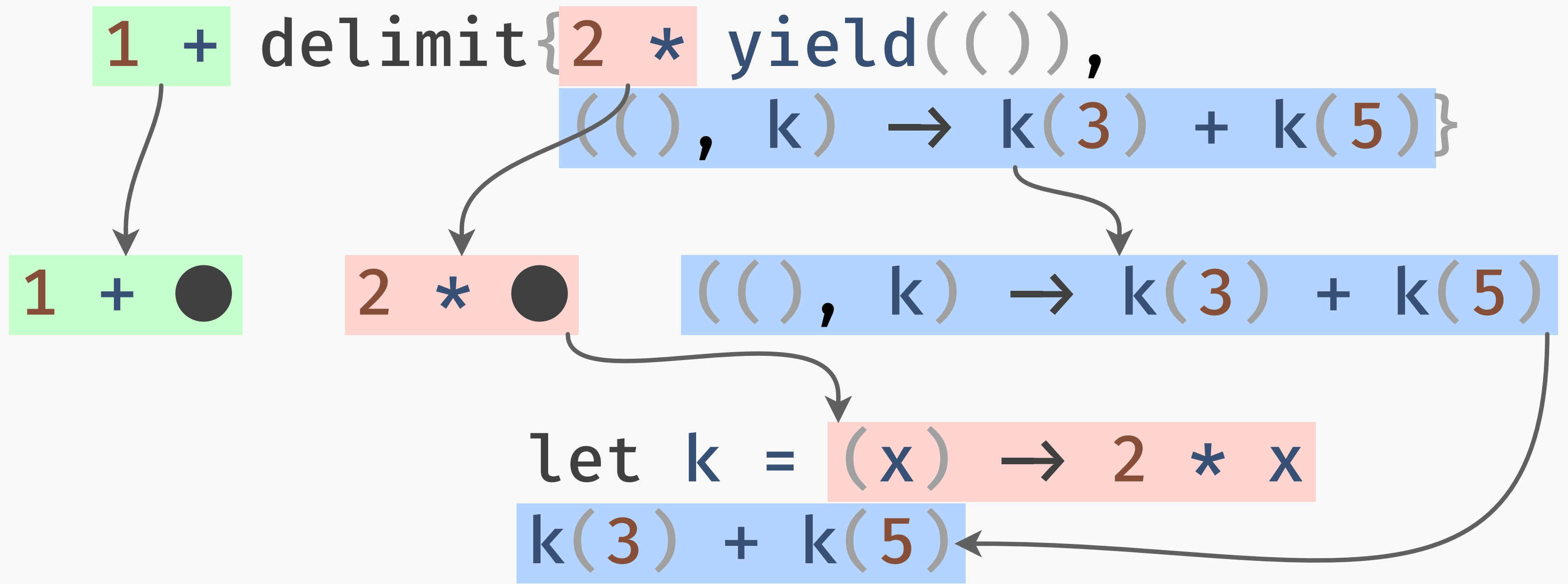


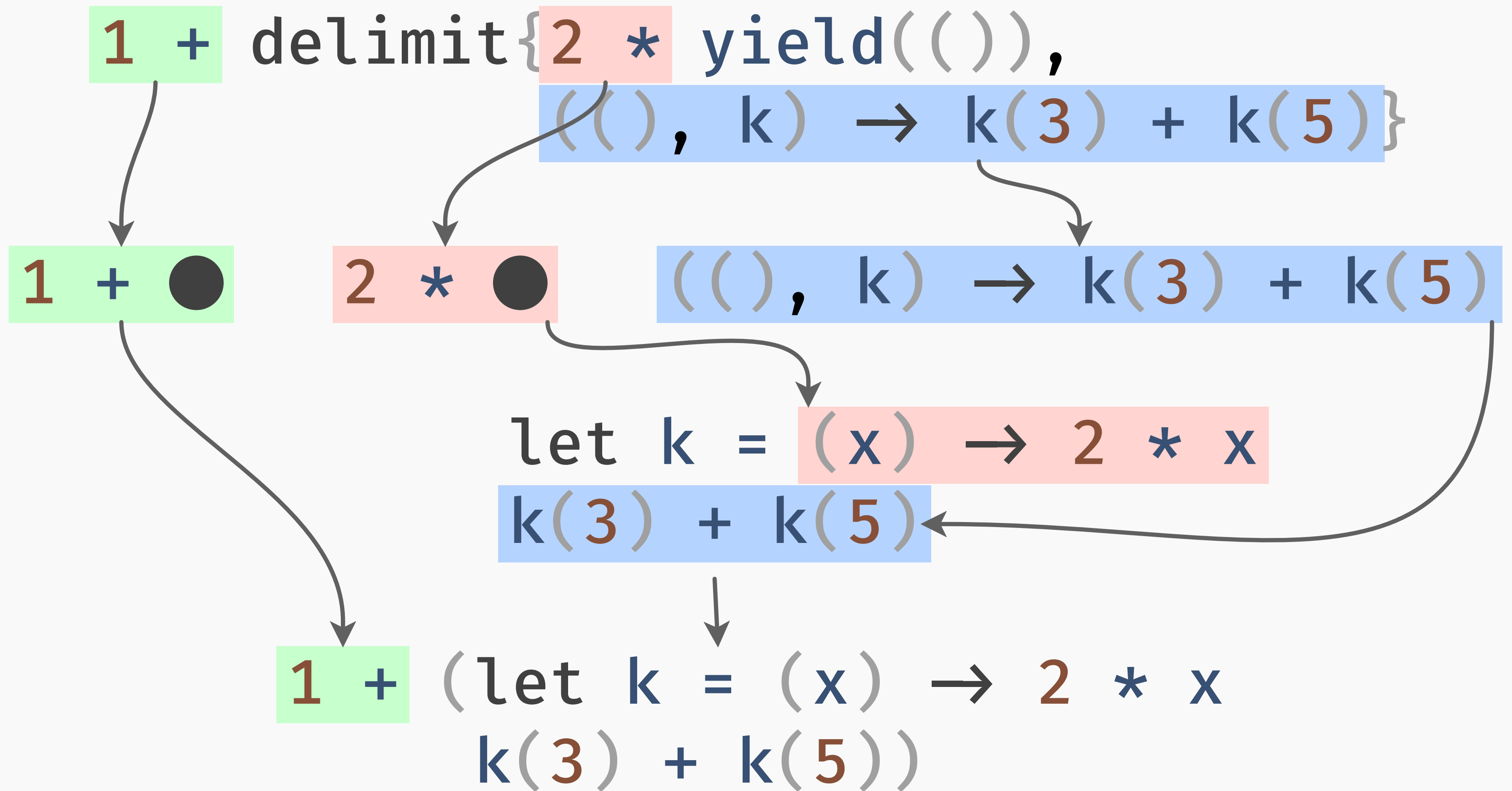
```
1 + delimit{2 * yield(()),  
            ((), k) → k(3) + k(5)}
```

```
1 + delimit{2 * yield(()),  
            ((), k) → k(3) + k(5)}
```

```
1 + delimit{2 * yield(()),  
            ((), k) → k(3) + k(5)}
```







Why prompt / control?

Why prompt / control?

→ In some sense “simpler”.

Why prompt / control?

- In some sense “simpler”.
- Historical relationship to `call_cc`.

Why prompt / control?

- In some sense “simpler”.
- Historical relationship to `call_cc`.
- Easier to statically type.

TYPES

Even typing exceptions is hard!

Even typing exceptions is hard!

throw : Exception → a

Even typing exceptions is hard!

throw : Exception → a

Even typing exceptions is hard!

throw : **Exception** $\rightarrow$ **a**

Even typing exceptions is hard!

`throw : Exception → a`

`catch{body, handler} : b`

Even typing exceptions is hard!

throw : **Exception** $\rightarrow$ **a**

catch{**body**, **handler**} : **b**

body : **b**

handler : **Exception** $\rightarrow$ **b**

Even typing exceptions is hard!

`throw : Exception → a`

`catch{body, handler} : b`

`body : b`

`handler : Exception → b`

Even typing exceptions is hard!

throw : **Exception** $\rightarrow$ **a**

catch{**body**, **handler**} : **b**

body : **b**

handler : **Exception** $\rightarrow$ **b**

Even typing exceptions is hard!

throw : **Exception** $\rightarrow$ **a**

catch{**body**, **handler**} : **b**

body : **b**

handler : **Exception** $\rightarrow$ **b**

`yield : DelimiterTag → a`

`yield : DelimiterTag → a`

`delimit{body, handler} : b`

`yield : DelimiterTag → a`

`delimit{body, handler} : b`

`body : b`

`handler : DelimiterTag → (a → b) → b`

`yield : DelimiterTag → a`

`delimit{body, handler} : b`

`body : b`

`handler : DelimiterTag → (a → b) → b`

`yield : DelimiterTag → a`

`delimit{body, handler} : b`

`body : b`

`handler : DelimiterTag → (a → b) → b`

$$E_1[\text{delimit}\{E_2[\text{yield}(v)], f\}]$$
$$\longrightarrow E_1[f(v, (x) \rightarrow E_2[x])]$$

`yield : DelimiterTag → a`

`delimit{body, handler} : b`

`body : b`

`handler : DelimiterTag → (a → b) → b`

$$E_1[\text{delimit}\{E_2[\text{yield}(v)], f\}]$$
$$\longrightarrow E_1[f(v, (x) \rightarrow E_2[x])]$$

`yield : DelimiterTag → a`

`delimit{body, handler} : b`
`body : b`

`handler : DelimiterTag → (a → b) → b`

$E_1[\text{delimit}\{E_2[\text{yield}(v)], f\}]$
 $\longrightarrow E_1[f(v, (x) \rightarrow E_2[x])]$

$\text{yield} : \text{DelimiterTag} \rightarrow a$

$\text{delimit}\{\text{body}, \text{handler}\} : b$

$\text{body} : b$

$\text{handler} : \text{DelimiterTag} \rightarrow (a \rightarrow b) \rightarrow b$

$$E_1[\text{delimit}\{E_2[\text{yield}(v)], f\}]$$
$$\longrightarrow E_1[f(v, (x) \rightarrow E_2[x])]$$

`yield : DelimiterTag → a`

`delimit{body, handler} : b`
`body : b`

`handler : DelimiterTag → (a → b) → b`

$E_1[\text{delimit}\{E_2[\text{yield}(v)], f\}]$
 $\longrightarrow E_1[f(v, (x) \rightarrow E_2[x])]$

`prompt{body} : b`

prompt{body} : b

body : b

prompt{body} : b

body : b

control : ((a → b) → b) → a

prompt{body} : b

body : b

control : ((a → b) → b) → a

$E_1[\text{prompt}\{E_2[\text{control}(f)]\}]$

$\longrightarrow E_1[f((\mathbf{x}) \rightarrow E_2[\mathbf{x}])]$

prompt{body} : b

body : b

control : ((a → b) → b) → a

$E_1[\text{prompt}\{E_2[\text{control}(f)]\}]$

$\longrightarrow E_1[f((x) \rightarrow E_2[x])]$

prompt{body} : b

body : b

control : ((**a** → b) → b) → **a**

$E_1[\text{prompt}\{E_2[\text{control}(f)]\}]$

$\longrightarrow E_1[f((\mathbf{x}) \rightarrow E_2[\mathbf{x}])]$

prompt{body} : **b**

body : **b**

control : ((a → **b**) → **b**) → a

$E_1[\text{prompt}\{E_2[\text{control}(f)]\}]$

$\longrightarrow E_1[f((\mathbf{x}) \rightarrow E_2[\mathbf{x}])]$

Solution: tagged prompts.

new_prompt_tag : () → PromptTag

`new_prompt_tag : () → PromptTag<b>`

`prompt{tag, body} : b`

new_prompt_tag : () → PromptTag

prompt{tag, body} : b

tag : PromptTag

body : b

$\text{new_prompt_tag} : () \rightarrow \text{PromptTag}\langle b \rangle$

$\text{prompt}\{\text{tag}, \text{body}\} : b$

$\text{tag} : \text{PromptTag}\langle b \rangle$

$\text{body} : b$

$\text{control} : (\text{PromptTag}\langle b \rangle, ((a \rightarrow b) \rightarrow b)) \rightarrow a$

new_prompt_tag : $() \rightarrow \text{PromptTag}\langle b \rangle$

prompt{tag, body} : b

tag : $\text{PromptTag}\langle b \rangle$

body : b

control : $(\text{PromptTag}\langle b \rangle, ((a \rightarrow b) \rightarrow b)) \rightarrow a$

$E_1[\text{prompt}\{tag, E_2[\text{control}(tag, f)]\}]$

$\longrightarrow E_1[f((x) \rightarrow E_2[x])]$

$\text{new_prompt_tag} : () \rightarrow \text{PromptTag}\langle b \rangle$

$\text{prompt}\{tag, body\} : b$

$tag : \text{PromptTag}\langle b \rangle$

$body : b$

$\text{control} : (\text{PromptTag}\langle b \rangle, ((a \rightarrow b) \rightarrow b)) \rightarrow a$

$E_1[\text{prompt}\{tag, E_2[\text{control}(tag, f)]\}]$

$\longrightarrow E_1[f((x) \rightarrow E_2[x])]$

$\text{new_prompt_tag} : () \rightarrow \text{PromptTag}\langle b \rangle$

$\text{prompt}\{tag, body\} : b$

$tag : \text{PromptTag}\langle b \rangle$

$body : b$

$\text{control} : (\text{PromptTag}\langle b \rangle, ((a \rightarrow b) \rightarrow b)) \rightarrow a$

$E_1[\text{prompt}\{tag, E_2[\text{control}(tag, f)]\}]$

$\longrightarrow E_1[f((x) \rightarrow E_2[x])]$

① continuations ✓

② delimited ✓

③ first-class

④ native

① continuations ✓

② delimited ✓

③ first-class ✓

④ native

① continuations ✓

② delimited ✓

③ first-class ✓

④ native

How do we implement this?

How do we implement this?

Option one: continuation-passing style.

How do we implement this?

Option one: continuation-passing style.

Problem: slow! (See my talk from ZuriHac 2020.)

How do we implement this?

Option one: continuation-passing style.

Problem: slow! (See my talk from ZuriHac 2020.)

Option two: bake them into the runtime.

1 + prompt{tag, f(true, ●) * 5}

1 + prompt{tag, f(true, ●) * 5}



1 + prompt{tag, f(true, ●) * 5}



This is a call stack!

redex: `control(tag, g)`

stack: `f(true, ●)`

● * 5

`prompt{tag, ●}`

1 + ●

redex: `control(tag, g)`

stack: `f(true, ●)`

● * 5

`prompt{tag, ●}`

1 + ●

redex: `control(tag, g)`

stack:

`f(true, ●)`

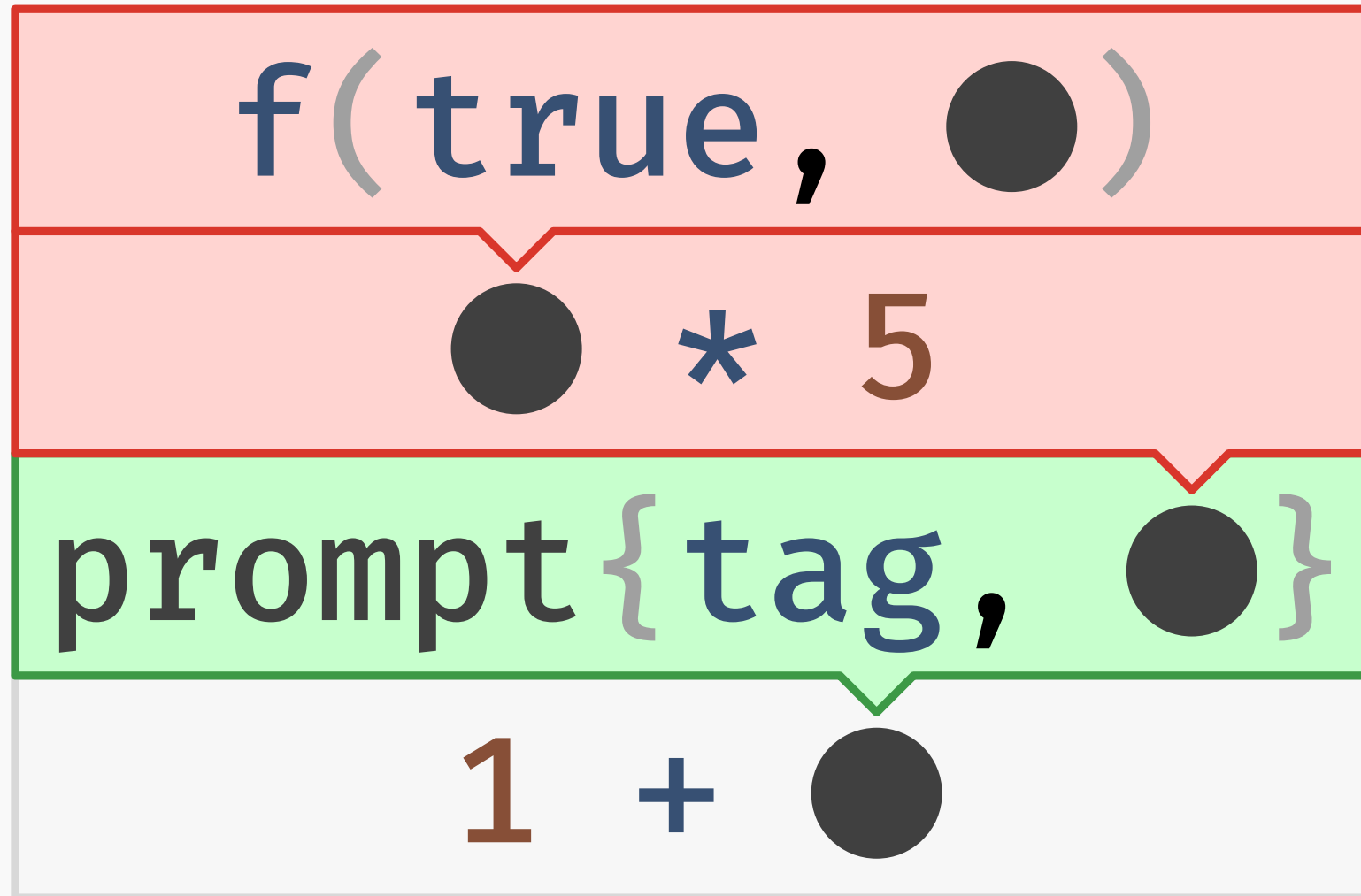
● * 5

`prompt{tag, ●}`

1 + ●

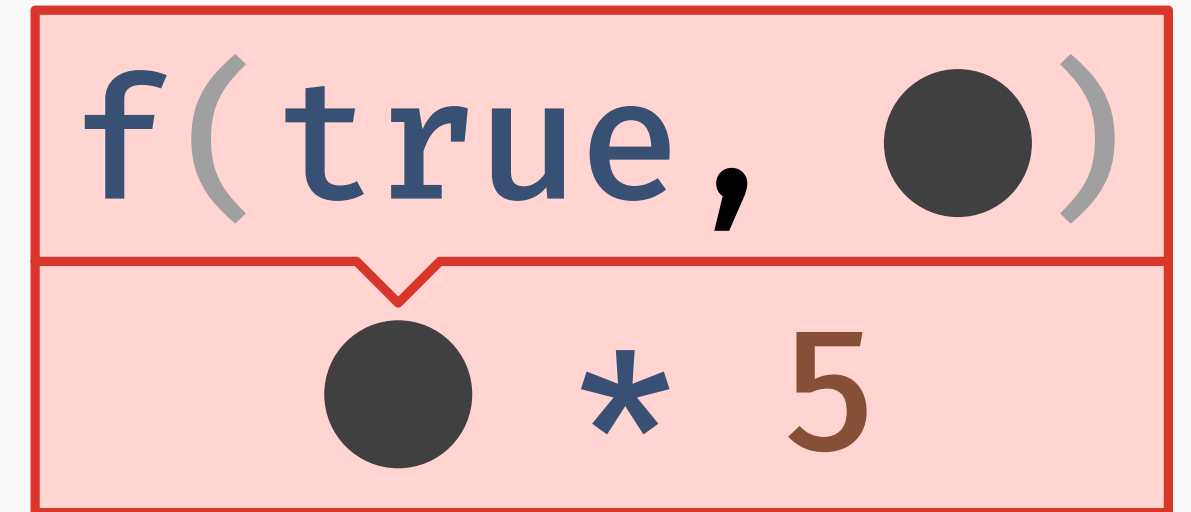
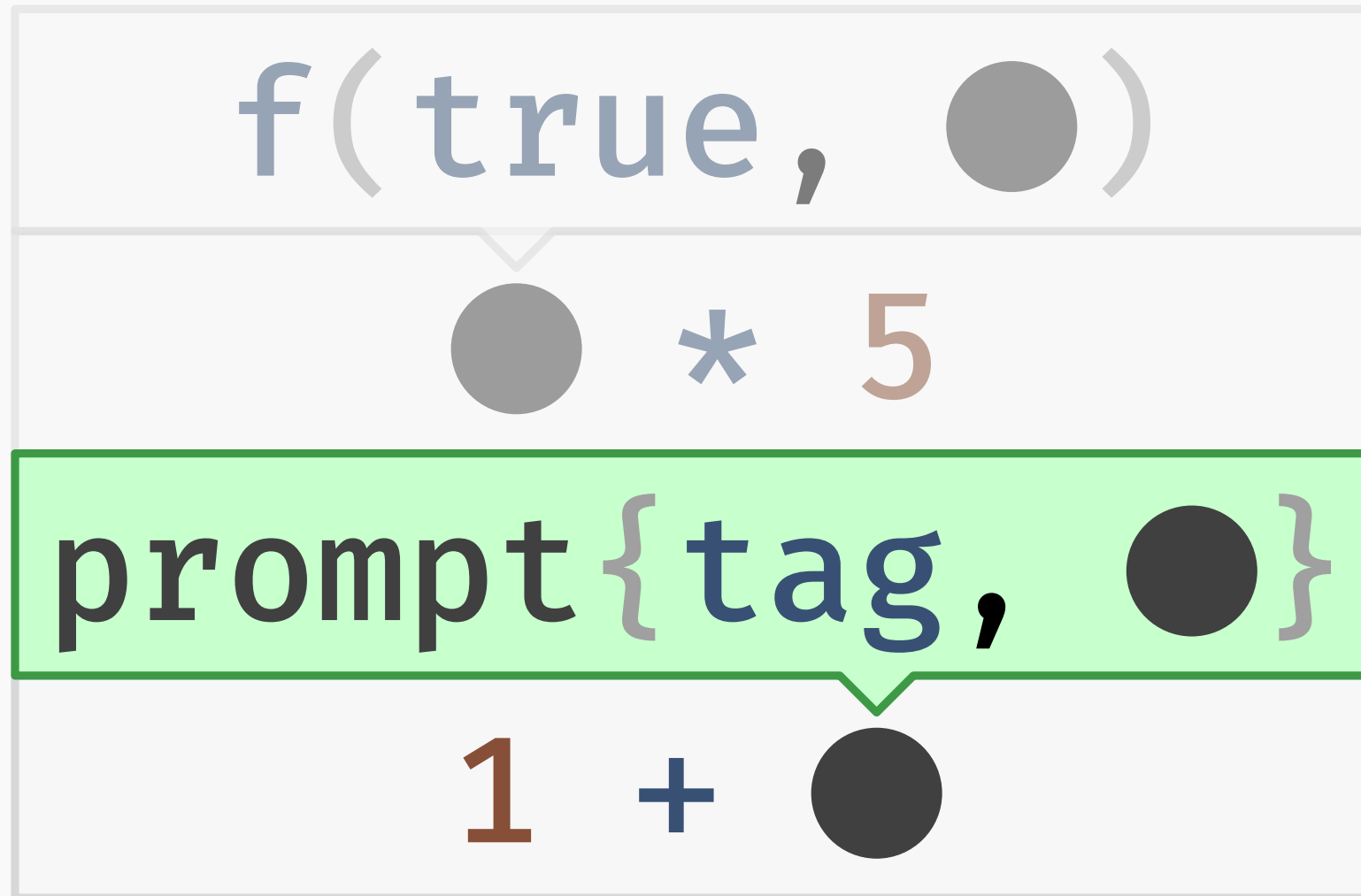
redex: `control(tag, g)`

stack:



redex: `control(tag, g)`

stack:



redex: `control(tag, g)`

stack: `prompt{tag, ●}`

`1 + ●`

`f(true, ●)`

`● * 5`

redex: **control**(tag, g)

stack: prompt{tag, ●}

1 + ●

f(true, ●)

● * 5

redex: `control(tag, g)`

stack:

`1 + ●`

`f(true, ●)`

`● * 5`

redex: $g(\text{CONT } \circ)$

stack:

$1 + \bullet$

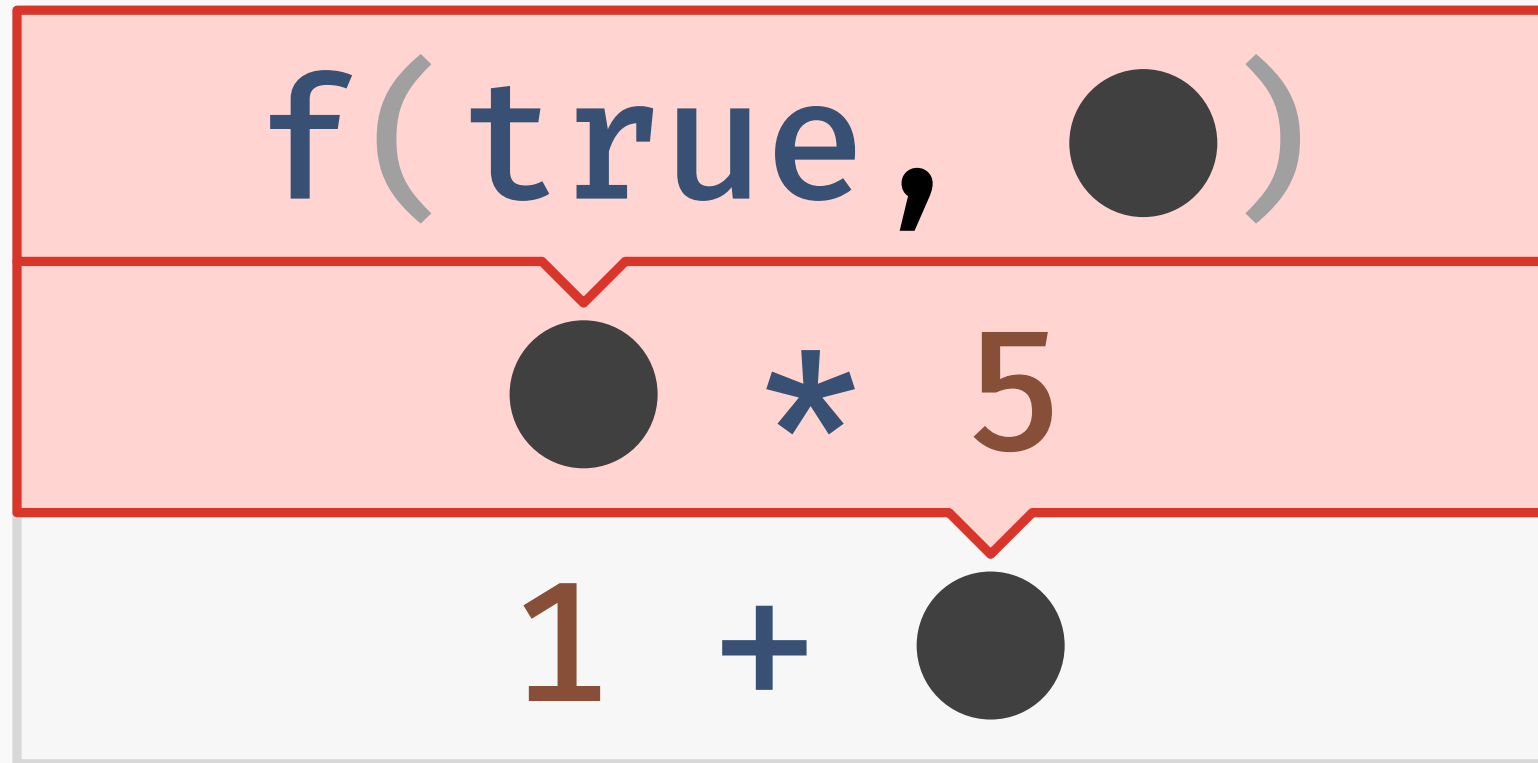
$f(\text{true}, \bullet)$

$\bullet * 5$



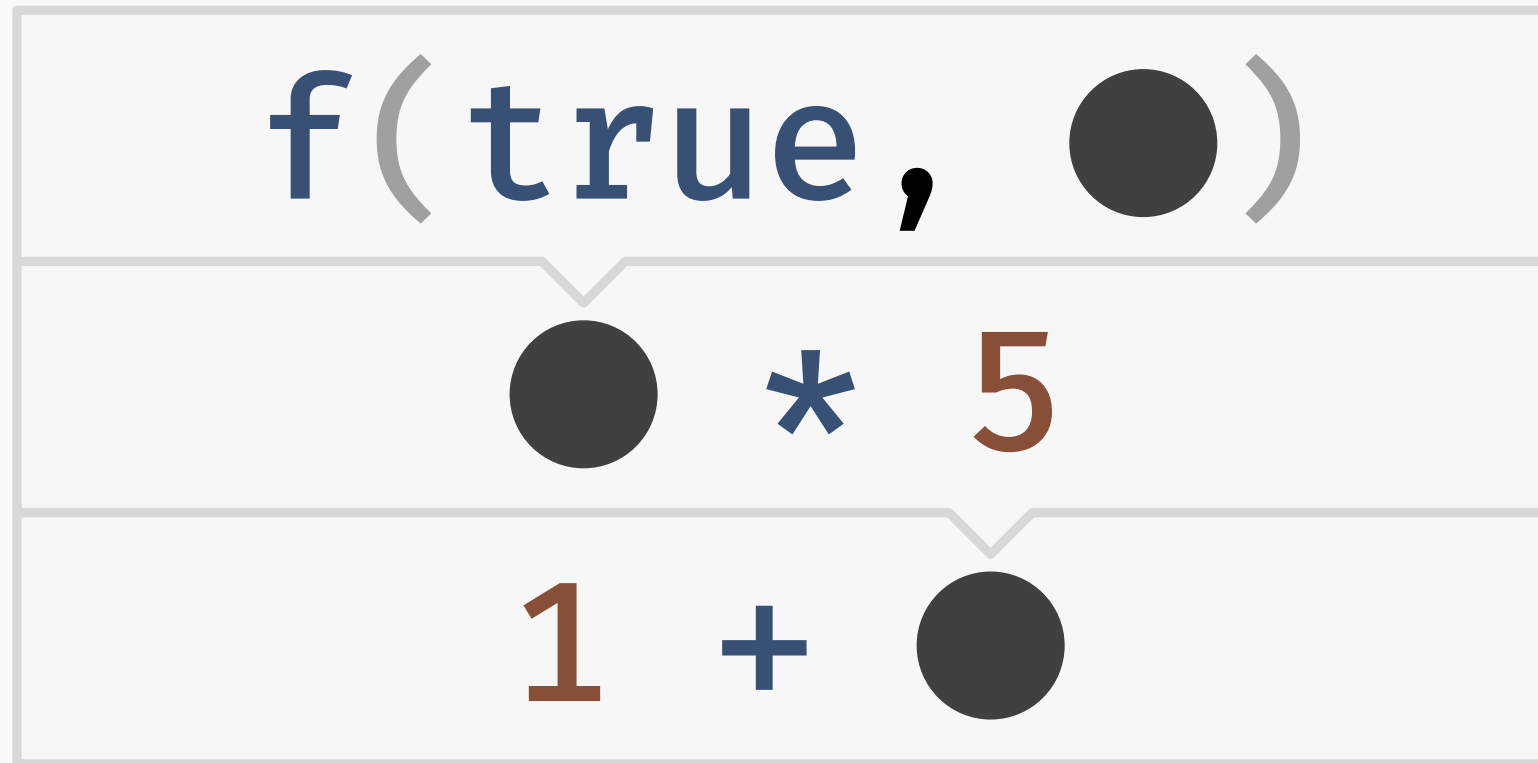
redex: "hello"

stack:



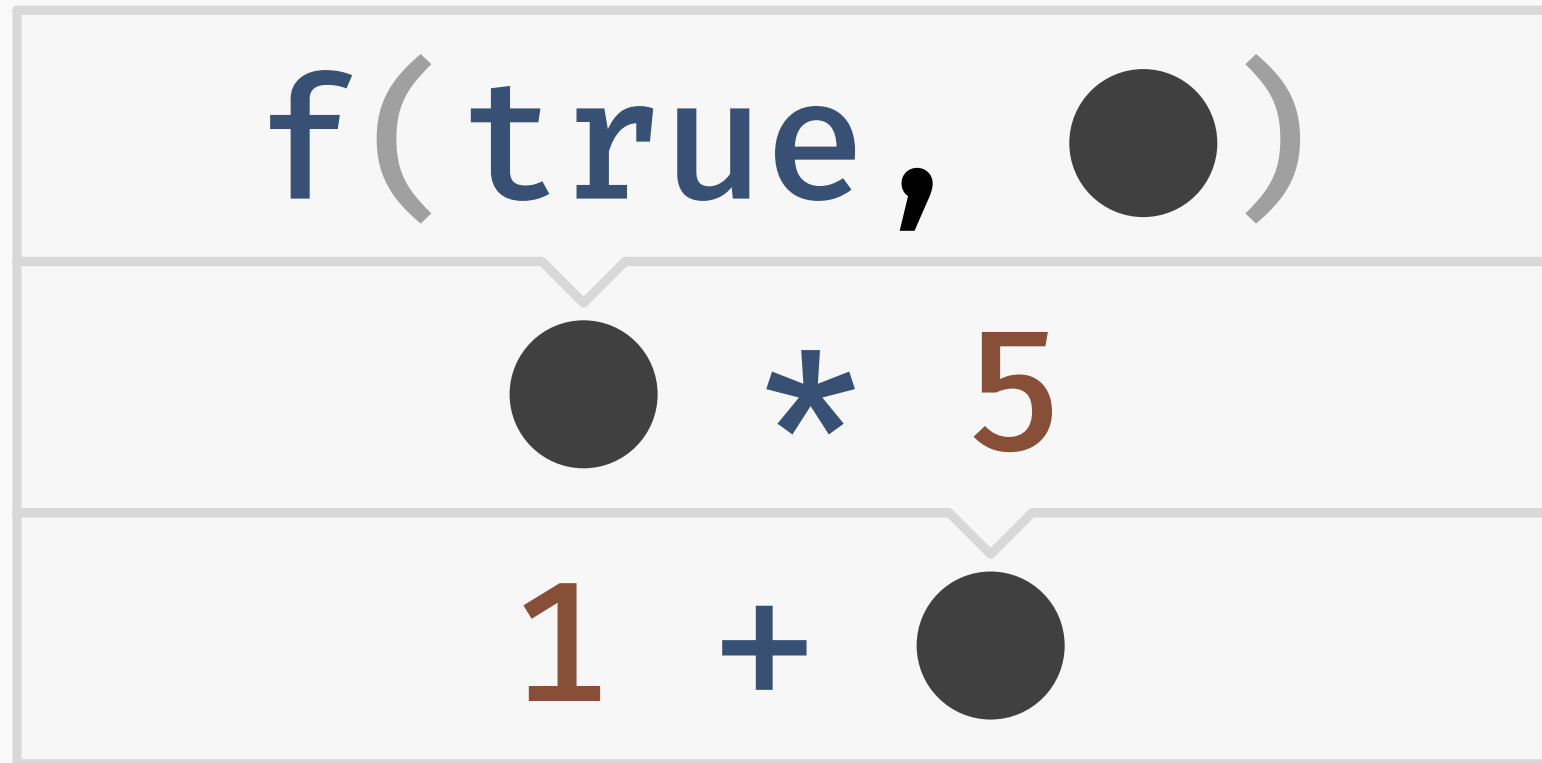
redex: "hello"

stack:



redex: "hello"

stack:



Capture/restore are just **memcpy**!

① continuations ✓

② delimited ✓

③ first-class ✓

④ native

① continuations ✓

② delimited ✓

③ first-class ✓

④ native ✓

MISCELLANY

MISCELLANY

→ Can further optimize implementation for specific use cases.

MISCELLANY

- Can further optimize implementation for specific use cases.
- Strict monads permit embedding into a lazy language.

MISCELLANY

- Can further optimize implementation for specific use cases.
- Strict monads permit embedding into a lazy language.
- Reality is always at least a little more complicated (e.g. stack overflow, async exceptions).

MISCELLANY

- Can further optimize implementation for specific use cases.
- Strict monads permit embedding into a lazy language.
- Reality is always at least a little more complicated (e.g. stack overflow, async exceptions).
- We sorely lack non-synthetic continuation benchmarks!

The unsung hero of this talk:

The unsung hero of this talk:
reduction semantics.

- ① continuations
- ② delimited
- ③ first-class
- ④ native

- ① continuations ✓
- ② delimited ✓
- ③ first-class
- ④ native

- ① continuations ✓
- ② delimited ✓
- ③ first-class
- ④ native

Still extremely useful!

→ Continuations are a concept that arises naturally in evaluation.

- Continuations are a concept that arises naturally in evaluation.
- Special operators like **catch** delimit portions of the continuation.

- Continuations are a concept that arises naturally in evaluation.
- Special operators like **catch** delimit portions of the continuation.
- First-class continuations allow reifying the continuation as a function.

- Continuations are a concept that arises naturally in evaluation.
- Special operators like **catch** delimit portions of the continuation.
- First-class continuations allow reifying the continuation as a function.
- Remarkably, this corresponds to manipulation of the call stack.

- Continuations are a concept that arises naturally in evaluation.
- Special operators like **catch** delimit portions of the continuation.
- First-class continuations allow reifying the continuation as a function.
- Remarkably, this corresponds to manipulation of the call stack.

Thanks!

me: <https://lexi-lambda.github.io/>

https://twitter.com/lexi_lambda

Tweag: <https://www.tweag.io/>